

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT-HÀN

BÁO CÁO TỔNG KẾT
ĐỀ TÀI KHOA HỌC VÀ CÔNG NGHỆ CẤP CƠ SỞ

**NGHIÊN CỨU CÁC MÔ HÌNH GAN (GENERATIVE ADVERSARIAL
NETWORK) XỬ LÝ MẤT CÂN BẰNG DỮ LIỆU TRONG DỰ ĐOÁN
LỖI PHẦN MỀM**

Mã số: ĐHVH-2023-03

Chủ nhiệm đề tài: ThS. Hà Thị Minh Phương

Đà Nẵng, 12/2023

ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT-HÀN

BÁO CÁO TỔNG KẾT

ĐỀ TÀI KHOA HỌC VÀ CÔNG NGHỆ
CẤP CƠ SỞ

NGHIÊN CỨU CÁC MÔ HÌNH GAN (GENERATIVE ADVERSARIAL
NETWORK) XỬ LÝ MẤT CÂN BẰNG DỮ LIỆU TRONG DỰ ĐOÁN
LỖI PHẦN MỀM

Mã số: ĐHVH-2023-03

Xác nhận của cơ quan chủ trì đề tài

Chủ nhiệm đề tài

Đà Nẵng, 12/2023

Mục lục

Danh sách hình vẽ	3
Danh sách bảng	4
1 Tổng quan đề tài	1
2 Dự đoán lỗi phần mềm	5
2.1 Tổng quan về dự đoán lỗi phần mềm	5
2.2 Quy trình dự đoán lỗi phần mềm	6
2.3 Độ đo phần mềm	8
2.4 Lựa chọn đặc trưng	10
2.4.1 Lựa chọn đặc trưng dựa trên Filter	10
2.4.2 Lựa chọn đặc trưng dựa trên Wrapper	11
2.4.3 Lựa chọn đặc trưng dựa trên Embedded	11
2.5 Các mô hình học máy	12
3 Các nghiên cứu liên quan	13
4 Mô hình GAN cho lấy mẫu dữ liệu trong dự đoán lỗi phần mềm	14
4.1 Mô hình mạng GAN	14
4.2 VanillaGAN	15
4.3 Conditional GAN - CTGAN	16
4.4 Wasserstein GAN With Gradient Penalty - WGANGP	17
5 Thực nghiệm đề xuất	17
5.1 Thực nghiệm dựa trên kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Filter (Filter-based feature selection) và các kỹ thuật lấy mẫu GAN	18
5.2 Thực nghiệm dựa trên kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Wrapper (wrapper-based feature selection) và các kỹ thuật lấy mẫu GAN	18

5.3	Tập dữ liệu thực nghiệm	20
5.4	Độ đo đánh giá	21
5.5	Kết quả thực nghiệm	24
5.5.1	Kết quả thực nghiệm 1	25
5.5.2	Kết quả thực nghiệm 2	27
6	Kết luận	29
	References	31

Danh sách hình vẽ

1	Quy trình dự đoán lỗi phần mềm.	7
2	Mô hình kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Filter và các kỹ thuật lấy mẫu GAN.	19
3	Mô hình kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và các kỹ thuật lấy mẫu GAN.	20

THÔNG TIN KẾT QUẢ NGHIÊN CỨU

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT HÀN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do - Hạnh phúc

THÔNG TIN KẾT QUẢ NGHIÊN CỨU

1. Thông tin chung:

- Tên đề tài: **Nghiên cứu các mô hình generative adversarial network (gan) xử lý mất cân bằng dữ liệu trong dự đoán lỗi phần mềm.**
- Mã số: ĐHVH-2023-03
- Chủ nhiệm: HÀ THỊ MINH PHƯƠNG
- Thành viên tham gia:
- Cơ quan chủ trì: Khoa Khoa học máy tính – Trường ĐH CNTT&TT Việt - Hàn – Đại học Đà Nẵng
- Thời gian thực hiện: từ tháng 5/2023 đến tháng 12/2023

2. Mục tiêu:

Nghiên cứu lý thuyết:

- Nghiên cứu mô hình Generative adversarial networks (GANs)
- Nghiên cứu các biến thể GANs để áp dụng vào việc bằng dữ liệu: VanillaGAN, CTGAN, WGANP, CopulaGAN và TabularGAN
- Nghiên cứu các mô hình máy học thực hiện dự đoán lỗi

Nghiên cứu và áp dụng các biến thể của GANs nói trên để đưa vào xử lý mất cân bằng dữ liệu nhằm giải quyết vấn đề overfitting nâng cao độ chính xác của mô hình dự đoán lỗi. Đồng thời, tiến hành thực nghiệm để so sánh hiệu quả của các mô hình GANs trên và các kỹ thuật truyền thống như SMOTE, ADASYN, Border-SMOTE, Random Undersampling trong việc xử lý vấn đề mất cân bằng dữ liệu trong dự đoán lỗi phần mềm.

3. Tính mới và sáng tạo:

- Lỗi phần mềm sẽ tác động mạnh đến các hệ thống phần mềm trong quá trình phát triển cũng như quá trình triển khai.
- Để nâng cao chất lượng trong dự đoán lỗi phần mềm, tác giả đã nghiên cứu các kỹ thuật máy học, các kỹ thuật trong giai đoạn tiền xử lý dữ liệu để xây dựng các mô hình dự đoán. Tuy nhiên, trong bộ dữ liệu lỗi phần mềm, các mô-đun bị lỗi (loại thiểu số) đại diện ít hơn nhiều so với các mô-đun không bị lỗi (loại đa số),

đây là vấn đề mất cân bằng loại. Do vấn đề này, hiệu suất của các mô hình dự đoán lỗi bị cản trở nghiêm trọng. Nhiều nhà nghiên cứu đã cố gắng khắc phục vấn đề này của mô hình dự đoán lỗi phân mềm. Do đó, các kỹ thuật lấy mẫu dữ liệu như oversampling và under-sampling được sử dụng rộng rãi để giải quyết vấn đề mất cân bằng lớp trong dự đoán lỗi phân mềm. Các kỹ thuật oversampling tạo ra các mô-đun lớp thiếu số để tăng số mẫu của lớp thiếu số. Ngược lại, các kỹ thuật under-sampling mức loại bỏ một số mô-đun của lớp đa số để cân bằng tỷ lệ các mô-đun của lớp thiếu số và đa số. Tuy nhiên, một số nghiên cứu đã cho thấy các kỹ thuật sampling trên đã dẫn đến các mô hình dự đoán bị overfitting. Trong những năm gần đây, mô hình GANs (Generative adversarial networks) đã được nghiên cứu đề xuất để đưa vào xử lý vấn đề mất cân bằng dữ liệu và giải quyết vấn đề overfitting của mô hình dự đoán lỗi. Vì vậy, trong nghiên cứu này, chúng tôi tập trung nghiên cứu đến mô hình GAN và các biến thể để xử lý vấn đề mất cân bằng dữ liệu trong dự đoán lỗi từ đó nâng cao hiệu quả dự đoán của các mô hình.

4. Tóm tắt kết quả nghiên cứu:

- Nghiên cứu đặc trưng phân mềm (metrics), vấn đề mất cân bằng dữ liệu trong dự đoán lỗi phân mềm.
- Nghiên cứu các mô hình máy học trong dự đoán lỗi phân mềm.
- Nghiên cứu các kỹ thuật cân bằng dữ liệu như SMOTE, ADASYN, Border-SMOTE, RUS,...
- Nghiên cứu và áp dụng mô hình GANs và các biến thể để đưa vào cân bằng dữ liệu nhằm nâng cao hiệu suất của mô hình dự đoán lỗi.
- Tiến hành thực nghiệm với các mô hình của GANs cân bằng dữ liệu trên các tập dữ liệu trong kho dữ liệu Promise để đánh giá kết quả đạt được và so sánh các kỹ thuật cân bằng dữ liệu truyền thống.

5. Tên sản phẩm:

- Báo cáo tổng kết đề tài;
- Bài báo đăng trên hội thảo CITA 2022 “Dự đoán lỗi phân mềm sử dụng Convolution Neural Network”;

6. Hiệu quả, phương thức chuyển giao kết quả nghiên cứu và khả năng áp dụng:

- Về mặt giáo dục - đào tạo: phục vụ công tác giảng dạy, nghiên cứu.
- Về mặt khoa học: đóng góp đáng kể của đề tài là nghiên cứu mô hình mới thực hiện dự đoán lỗi bằng phân tích ngữ nghĩa của mã nguồn, qua đó có thể đề xuất được các kỹ thuật máy học có tính hiệu quả và chính xác cao hơn so với các mô hình dự đoán lỗi dựa trên độ đo mã nguồn.

THÔNG TIN KẾT QUẢ NGHIÊN CỨU

- Về sản phẩm ứng dụng: đề xuất mô hình dự đoán lỗi và hướng tới áp dụng được các hệ thống dự đoán được lỗi phần mềm thực tế trong công nghệ phần mềm.

7. Hình ảnh, sơ đồ minh họa chính:

Dataset	Performance	Chi-Squared											
	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.826	0.856	0.843	0.849	0.843	0.856	0.854	0.837	0.834	0.819	0.828	0.821
	Recall	0.712	0.824	0.780	0.810	0.860	0.878	0.848	0.846	0.804	0.832	0.726	0.816
	F1-score	0.758	0.834	0.806	0.827	0.849	0.861	0.849	0.840	0.816	0.824	0.766	0.817
	AUC	0.635	0.739	0.679	0.706	0.675	0.739	0.723	0.690	0.643	0.657	0.574	0.679
KC1	Precision	0.809	0.816	0.807	0.823	0.817	0.827	0.813	0.825	0.804	0.804	0.797	0.808
	Recall	0.759	0.826	0.802	0.831	0.845	0.848	0.841	0.850	0.814	0.808	0.815	0.802
	F1-score	0.778	0.820	0.804	0.826	0.820	0.830	0.817	0.830	0.803	0.802	0.801	0.799
	AUC	0.752	0.794	0.753	0.780	0.793	0.793	0.745	0.781	0.755	0.734	0.647	0.734
KC2	Precision	0.815	0.785	0.795	0.788	0.798	0.785	0.786	0.787	0.809	0.813	0.807	0.825
	Recall	0.809	0.798	0.798	0.796	0.815	0.806	0.8	0.806	0.638	0.672	0.657	0.668
	F1-score	0.811	0.788	0.794	0.788	0.792	0.788	0.788	0.791	0.665	0.698	0.682	0.693
	AUC	0.777	0.783	0.701	0.773	0.785	0.775	0.658	0.762	0.753	0.799	0.744	0.761
PC1	Precision	0.899	0.919	0.919	0.916	0.877	0.925	0.913	0.930	0.884	0.911	0.887	0.891
	Recall	0.824	0.916	0.907	0.916	0.922	0.935	0.921	0.938	0.817	0.814	0.682	0.823
	F1-score	0.854	0.917	0.911	0.915	0.897	0.925	0.916	0.929	0.837	0.836	0.739	0.844
	AUC	0.780	0.784	0.839	0.826	0.724	0.775	0.838	0.802	0.681	0.698	0.616	0.663
Average													
	Precision	0.837	0.844	0.841	0.844	0.833	0.848	0.841	0.845	0.833	0.837	0.830	0.836
	Recall	0.776	0.841	0.822	0.838	0.860	0.867	0.852	0.860	0.768	0.781	0.720	0.777
	F1-score	0.800	0.840	0.829	0.839	0.839	0.851	0.842	0.847	0.780	0.790	0.747	0.788
	AUC	0.736	0.775	0.743	0.771	0.744	0.771	0.741	0.759	0.708	0.722	0.645	0.709

Bảng 1. Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Chi-Squared and GAN

Dataset	Performance	Information Gain											
	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.853	0.853	0.843	0.844	0.832	0.863	0.864	0.832	0.834	0.823	0.827	0.820
	Recall	0.722	0.840	0.790	0.812	0.870	0.888	0.866	0.850	0.858	0.852	0.782	0.790
	F1-score	0.769	0.843	0.812	0.826	0.847	0.865	0.861	0.839	0.844	0.836	0.801	0.802
	AUC	0.671	0.695	0.644	0.658	0.671	0.695	0.662	0.627	0.633	0.63	0.621	0.638
KC1	Precision	0.797	0.808	0.811	0.808	0.815	0.807	0.810	0.809	0.814	0.795	0.804	0.802
	Recall	0.836	0.830	0.841	0.832	0.845	0.832	0.840	0.836	0.827	0.796	0.807	0.789
	F1-score	0.801	0.815	0.818	0.816	0.822	0.815	0.814	0.817	0.817	0.792	0.804	0.793
	AUC	0.765	0.784	0.735	0.761	0.773	0.778	0.737	0.754	0.763	0.725	0.664	0.725
KC2	Precision	0.803	0.807	0.791	0.811	0.821	0.803	0.776	0.794	0.798	0.804	0.813	0.812
	Recall	0.804	0.815	0.791	0.813	0.825	0.815	0.787	0.808	0.64	0.643	0.628	0.634
	F1-score	0.800	0.807	0.789	0.807	0.807	0.801	0.778	0.796	0.661	0.668	0.655	0.657
	AUC	0.777	0.791	0.726	0.792	0.812	0.788	0.651	0.783	0.765	0.788	0.737	0.767
PC1	Precision	0.893	0.918	0.899	0.907	0.881	0.924	0.909	0.911	0.879	0.890	0.878	0.889
	Recall	0.892	0.915	0.907	0.917	0.919	0.932	0.922	0.922	0.799	0.877	0.818	0.858
	F1-score	0.891	0.915	0.902	0.911	0.897	0.925	0.913	0.914	0.829	0.876	0.838	0.866
	AUC	0.772	0.755	0.757	0.797	0.689	0.747	0.781	0.780	0.649	0.663	0.559	0.698
Average													
	Precision	0.836	0.846	0.836	0.843	0.837	0.849	0.840	0.836	0.831	0.828	0.830	0.830
	Recall	0.813	0.850	0.832	0.844	0.864	0.867	0.854	0.854	0.781	0.792	0.759	0.768
	F1-score	0.816	0.845	0.830	0.840	0.843	0.852	0.842	0.842	0.788	0.793	0.775	0.780
	AUC	0.746	0.753	0.716	0.752	0.736	0.753	0.708	0.736	0.702	0.702	0.645	0.707

Bảng 2. Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Information Gain và GAN

THÔNG TIN KẾT QUẢ NGHIÊN CỨU

Dataset	Performance				Fisher				WGANGP				
	Evaluation	VanillaGAN				CTGAN							
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.833	0.842	0.836	0.828	0.845	0.837	0.836	0.840	0.818	0.814	0.828	HgB
	Recall	0.674	0.838	0.766	0.794	0.850	0.866	0.816	0.852	0.788	0.852	0.746	0.832
	F1-score	0.734	0.839	0.794	0.810	0.844	0.848	0.824	0.845	0.799	0.832	0.778	0.820
	AUC	0.632	0.678	0.615	0.689	0.670	0.707	0.619	0.659	0.613	0.653	0.637	0.824
KC1	Precision	0.809	0.827	0.813	0.828	0.818	0.833	0.813	0.829	0.803	0.800	0.806	0.708
	Recall	0.785	0.839	0.834	0.844	0.843	0.855	0.839	0.851	0.800	0.792	0.783	0.805
	F1-score	0.794	0.832	0.819	0.833	0.823	0.836	0.819	0.835	0.798	0.795	0.793	0.773
	AUC	0.763	0.798	0.749	0.770	0.783	0.798	0.748	0.778	0.755	0.747	0.686	0.785
KC2	Precision	0.795	0.810	0.794	0.825	0.784	0.792	0.795	0.794	0.818	0.827	0.806	0.755
	Recall	0.783	0.811	0.794	0.830	0.8	0.811	0.809	0.815	0.675	0.664	0.628	0.829
	F1-score	0.787	0.808	0.793	0.826	0.778	0.793	0.799	0.799	0.700	0.690	0.649	0.674
	AUC	0.774	0.793	0.708	0.763	0.781	0.787	0.666	0.761	0.724	0.803	0.728	0.699
PC1	Precision	0.902	0.919	0.927	0.920	0.893	0.930	0.925	0.935	0.890	0.903	0.880	0.732
	Recall	0.900	0.93	0.932	0.926	0.914	0.939	0.932	0.942	0.846	0.877	0.732	0.896
	F1-score	0.900	0.922	0.926	0.922	0.899	0.932	0.927	0.935	0.865	0.883	0.785	0.851
	AUC	0.867	0.808	0.817	0.889	0.749	0.816	0.867	0.879	0.700	0.732	0.611	0.868
Average													
	Precision	0.835	0.848	0.842	0.850	0.835	0.848	0.842	0.849	0.832	0.836	0.830	0.840
	Recall	0.786	0.855	0.832	0.849	0.852	0.867	0.849	0.865	0.777	0.796	0.722	0.780
	F1-score	0.804	0.850	0.833	0.848	0.836	0.852	0.842	0.853	0.790	0.800	0.751	0.794
	AUC	0.759	0.769	0.722	0.778	0.746	0.776	0.725	0.769	0.698	0.734	0.666	0.733

Bảng 3. Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Fisher và GAN

Dataset	Performance					Relief							
	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
3-14	Precision	0.845	0.848	0.843	0.835	0.824	0.872	0.855	0.844	0.829	0.834	0.839	0.831
	Recall	0.696	0.808	0.784	0.788	0.866	0.890	0.860	0.866	0.828	0.864	0.752	0.814
	F1-score	0.75	0.824	0.807	0.807	0.844	0.868	0.856	0.851	0.827	0.848	0.787	0.821
	AUC	0.647	0.716	0.672	0.673	0.676	0.711	0.716	0.670	0.648	0.637	0.661	0.690
CM1	Precision	0.815	0.824	0.811	0.827	0.815	0.833	0.825	0.843	0.815	0.821	0.816	0.824
	Recall	0.781	0.838	0.833	0.845	0.844	0.854	0.85	0.861	0.811	0.806	0.782	0.800
	F1-score	0.793	0.829	0.819	0.833	0.821	0.836	0.828	0.847	0.810	0.807	0.792	0.805
	AUC	0.773	0.798	0.737	0.776	0.788	0.797	0.757	0.788	0.759	0.746	0.720	0.740
KC1	Precision	0.811	0.789	0.789	0.798	0.781	0.802	0.785	0.789	0.816	0.809	0.797	0.812
	Recall	0.794	0.804	0.791	0.804	0.804	0.817	0.798	0.806	0.658	0.651	0.609	0.642
	F1-score	0.800	0.793	0.788	0.799	0.78	0.800	0.789	0.792	0.682	0.675	0.634	0.665
	AUC	0.796	0.789	0.705	0.772	0.781	0.783	0.652	0.772	0.780	0.791	0.738	0.760
KC2	Precision	0.900	0.913	0.916	0.915	0.871	0.922	0.914	0.929	0.893	0.902	0.893	0.898
	Recall	0.859	0.917	0.905	0.913	0.914	0.932	0.921	0.937	0.794	0.770	0.695	0.759
	F1-score	0.873	0.914	0.909	0.912	0.89	0.921	0.915	0.928	0.823	0.798	0.747	0.800
	AUC	0.782	0.783	0.839	0.814	0.691	0.775	0.841	0.802	0.680	0.707	0.646	0.685
Average													
3-14	Precision	0.843	0.843	0.840	0.844	0.822	0.857	0.845	0.851	0.838	0.841	0.836	0.841
	Recall	0.783	0.842	0.828	0.837	0.857	0.873	0.857	0.867	0.773	0.773	0.710	0.753
	F1-score	0.804	0.840	0.831	0.838	0.834	0.856	0.847	0.855	0.785	0.782	0.740	0.773
	AUC	0.757	0.767	0.738	0.759	0.734	0.767	0.742	0.758	0.717	0.720	0.691	0.719

Bảng 4. Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Relief và GAN

Đà Nẵng, ngày tháng năm 2023

Cơ quan chủ trì

Chủ nhiệm đề tài

Hà Thị Minh Phương

MỞ ĐẦU

I. TỔNG QUAN TÌNH HÌNH NGHIÊN CỨU THUỘC LĨNH VỰC ĐỀ TÀI TRONG VÀ NGOÀI NƯỚC

1. Ngoài nước

a. Dự đoán lỗi phần mềm

Trong bộ dữ liệu lỗi phần mềm, các mô-đun bị lỗi (loại thiểu số) đại diện ít hơn nhiều so với các mô-đun không bị lỗi (loại đa số), đây là vấn đề mất cân bằng loại. Do vấn đề này, hiệu suất của các mô hình dự đoán lỗi bị cản trở nghiêm trọng. Nhiều nhà nghiên cứu đã cố gắng khắc phục vấn đề này của mô hình dự đoán lỗi phần mềm. Do đó, các kỹ thuật lấy mẫu dữ liệu như oversampling và under-sampling được sử dụng rộng rãi để giải quyết vấn đề mất cân bằng lớp trong dự đoán lỗi phần mềm. Các kỹ thuật oversampling tạo ra các mô-đun lớp thiểu số để tăng số mẫu của lớp thiểu số. Ngược lại, các kỹ thuật under-sampling mức loại bỏ một số mô-đun của lớp đa số để cân bằng tỷ lệ các mô-đun của lớp thiểu số và đa số. Kameiet al.[1] đã đánh giá các phương pháp lấy mẫu khác nhau (ROS, SMOTE, RUS và lựa chọn một phía) cho dự đoán lỗi bằng các kỹ thuật máy học khác nhau trong công việc của họ. Các tác giả đã chỉ ra rằng các phương pháp lấy mẫu giúp cải thiện hiệu suất của các mô hình logistic và tuyến tính, trong khi các mô hình cây phân loại và mạng thần kinh hoạt động kém hiệu quả. Trong một nghiên cứu tương tự khác, Tanet al.[2] đã đánh giá bốn phương pháp lấy mẫu (simple dupli-cate, spread subsample, SMOTE, and resampling with/withoutr eplacement). Kết quả thử nghiệm cho thấy rằng phương pháp resampling được thực hiện tốt nhất trên toàn thể giới và các phương pháp sampling đã cải thiện hiệu suất dự đoán. Các phương pháp ensemble giải quyết vấn đề mất cân bằng lớp bằng cách tạo và kết hợp một số mô hình yếu để có được mô hình mạnh. Các phương pháp tập hợp được sử dụng phổ biến nhất là bag-ging, boost và stacking. Một số biến thể như Under-Bagging, OverBagging, UnderOverBagging, SMOTEBagging, RUBoost và SMOTEBoost [3,4] đã được áp dụng cho dự đoán lỗi. Tuy nhiên, một số nghiên cứu đã cho thấy các kỹ thuật sampling trên đã dẫn đến các mô hình dự đoán bị overfitting. Trong những năm gần đây, mô hình GANs (Generative adversarial networks) đã được nghiên cứu đề xuất để đưa vào xử lý vấn đề mất cân bằng dữ liệu và giải quyết vấn đề overfitting của mô hình dự đoán lỗi. Vì vậy, trong nghiên cứu này, chúng tôi tập trung nghiên cứu đến mô hình GAN và các biến thể để xử lý vấn đề mất cân bằng dữ liệu trong dự đoán lỗi từ đó nâng cao hiệu quả dự đoán của các mô hình.

[1] Kamei, A. Monden, S. Matsumoto, T. Kakimoto, and K.-I. Matsumoto, "The effects of over and under sampling on fault-prone module detection," in Proc. 1st Int. Symp. Empirical Softw. Eng. Meas., 2007, pp. 196–204

[2] M. Tan, L. Tan, S. Dara, and C. Mayeux, “Online defect prediction for imbalanced data,” in Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng., 2015, pp. 99–108.

[3] C. Seiffert, T. M. Khoshgoftaar, J. Van Hulse, and A. Napolitano, “RUS-boost: A hybrid approach to alleviating class imbalance,” IEEE Trans. Syst., Man, Cybern.-Part A, Syst. Humans, vol. 40, no. 1, pp. 185–197, Jan. 2010.

[4] Z. Sun, Q. Song, and X. Zhu, “Using coding-based ensemble learning to improve software defect prediction,” IEEE Trans. Syst., Man, Cybern., Part C, vol. 42, no. 6, pp. 1806–1817, Nov. 2012.

2. Trong nước

Dự đoán lỗi phần mềm là một trong những giai đoạn quan trọng để đảm bảo chất lượng của phần mềm. Dự đoán lỗi phần mềm có thể xác định mô-đun lỗi trong phần mềm và phân loại các lỗi. Các kiểm thử có thể tập trung kiểm tra các mô-đun được dự đoán lỗi lỗi đó trước. Các dự án dự đoán lỗi truyền thống tập trung trên xây dựng các mô hình với các kỹ thuật máy học và độ đo (metrics). Các kỹ thuật học máy chủ yếu đã được áp dụng để phát triển các mô hình dự đoán lỗi phần mềm. Các kỹ thuật này sử dụng dữ liệu lỗi phần mềm lịch sử (thu được từ các dự án phần mềm trước đó) để huấn luyện các mô hình dự đoán và dự đoán các mô-đun bị lỗi. Trong quá trình huấn luyện, mô hình dự đoán thu được kiến thức về các đặc trưng của dự án phần mềm và sử dụng kiến thức này để dự đoán liệu mô-đun trong dự án khác có chứa lỗi hay không. Tuy nhiên, hầu hết các bộ dữ liệu lỗi lịch sử bao gồm mô-đun không bị lỗi (tức là lớp đa số) nhiều hơn là các mô-đun bị lỗi (tức là lớp thiểu số). Đây được gọi là vấn đề cân bằng lớp trong dự đoán lỗi phần mềm. Nhiều kỹ thuật học máy giả định rằng tỷ lệ của các mô-đun lớp thiểu số và mô-đun lớp đa số là xấp xỉ bằng nhau. Tuy nhiên, mô hình dự đoán được xây dựng trên tập dữ liệu phần mềm mất cân bằng có khả năng không học được các mẫu mô-đun bị lỗi và do đó có xu hướng dự đoán sai nhiều mô-đun bị lỗi. Điều này có thể dẫn đến các mô hình dự đoán không đáng tin cậy và không thể sử dụng trong thực tế. Vì vậy xử lý mất cân bằng dữ liệu trong dự đoán lỗi phần mềm đã được tập trung nghiên cứu trong những năm qua. Nhiều nghiên cứu đã đề xuất các kỹ thuật xử lý mất cân bằng dữ liệu như data oversampling và data undersampling. Trong quá trình tìm tài liệu tham khảo bằng tiếng Việt về dự đoán lỗi phần mềm, chúng tôi chỉ tìm thấy một bài báo thử nghiệm 3 kỹ thuật xử lý dữ liệu mất cân bằng dữ liệu [5] gồm: Random Undersampling, Random Oversampling và SMOTE. Các kỹ thuật được áp dụng vào ba tập dữ liệu về dự báo lỗi của NASA trong kho lưu trữ của Promise.

[5] Lê, Song Toàn, Thanh Bình Nguyễn, and Thị Mỹ Hạnh Lê, “Xử lý dữ liệu không cân bằng trong bài toán dự đoán lỗi phần mềm”, Kỷ yếu Hội nghị KHCN Quốc gia lần thứ XIII về Nghiên cứu cơ bản và ứng dụng Công nghệ thông tin (FAIR), 2020.

II. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Cả hai bộ kỹ thuật Oversampling và Undersampling nhằm mục đích cân bằng việc biểu diễn các mô-đun lớp thiểu số và đa số để cải thiện hiệu suất của các mô hình dự đoán. Đối với các kỹ thuật oversampling trong dự đoán lỗi phần mềm, các kỹ thuật như SMOTE và biến thể của nó (ví dụ: ADASYN, Border-line_SMOTE) và Random Oversampling (ROS) hầu như được sử dụng. Kỹ thuật ROS chỉ đơn giản là sao chép các mô-đun lớp thiểu số để cân bằng tập dữ liệu mà không cần thêm bất kỳ thông tin mới nào [6]. Điều này dẫn đến một mô hình dự đoán bị overfitting, có thể không hoạt động chính xác đối với các mô-đun thử nghiệm khác. Các kỹ thuật dựa trên SMOTE tạo ra các mô-đun lớp thiểu số tổng hợp bằng cách nội suy các mô-đun lân cận đã xác định (dựa trên khoảng cách) để cân bằng tập dữ liệu [7,8]. Các mô-đun mới được thêm này không phải là các mô-đun trùng lặp như trong ROS, nhưng do các mô-đun lân cận, các kỹ thuật dựa trên SMOTE không thể tạo ra các mô-đun lớp thiểu số đa dạng, điều này có thể dẫn đến việc overfitting của nhiều mô hình dự đoán (hiệu suất thấp trên tập hợp con thử nghiệm). Vì vậy, chúng tôi tiến hành nghiên cứu về các mô hình GAN để thực hiện việc oversampling dữ liệu có tính đa dạng nhằm giải quyết vấn đề mất cân bằng lớp trong các mô hình dự đoán lỗi.

[6] M. Hayaty, S. Muthmainah, and S. M. Ghufuran, “Random and synthetic over-sampling approach to resolve data imbalance in classification,” *Int.J. Artif. Intell. Res.*, vol. 4, no. 2, pp. 86–94, 2021.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[8] H. Han, W.-Y. Wang, and B.-H. Mao, “Borderline-smote: A new over-sampling method in imbalanced data sets learning,” in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.

[9] H. He, Y. Bai, E. A. Garcia, and S. Li, “ADASYN: Adaptive synthetic sampling approach for imbalanced learning,” in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2008, pp. 1322–1328.

III. MỤC TIÊU CỦA ĐỀ TÀI

Nghiên cứu mạng GANs và các biến thể của GANs như VanillaGAN, CTGAN, WGANP, CopullaGAN và TabularGAN để tạo ra các mẫu dữ liệu gần giống với phân phối dữ liệu lớp thiểu số ban đầu tương tự như các kỹ thuật sampling truyền thống, chẳng hạn như SMOTE. Tính đa dạng cũng có thể đạt được khi các mẫu đã tạo được chọn dựa trên phân phối dữ liệu thay vì dựa trên khoảng cách gần để sinh ra các mẫu dữ liệu. Từ đó, áp dụng các biến thể của GANs nói trên để đưa vào xử lý mất cân bằng dữ liệu nhằm giải quyết vấn đề overfitting nâng cao độ chính xác của mô hình dự đoán lỗi. Đồng thời, tiến hành thực nghiệm để so sánh hiệu quả của các mô hình GANs trên và các kỹ thuật truyền thống như SMOTE, ADASYN, Border-SMOTE, Random Undersampling trong việc xử lý vấn đề mất cân bằng dữ liệu trong dự đoán lỗi phần mềm.

Nghiên cứu lý thuyết:

- ✓ Nghiên cứu mô hình Generative adversarial networks (GANs)

- ✓ Nghiên cứu các biến thể GANs để áp dụng vào việc bằng dữ liệu:
VanillaGAN, CTGAN, WGANGP, CopullaGAN và TabularGAN
- ✓ Nghiên cứu các mô hình máy học thực hiện dự đoán lỗi

IV. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

1. Đối tượng nghiên cứu

- Các kỹ thuật Oversampling và Undersampling để xử lý mất cân bằng dữ liệu trong dự đoán lỗi phần mềm.
- Mô hình Generative adversarial networks và các biến thể VanillaGAN, CTGAN, WGANGP, CopullaGAN và TabularGAN.
- Các mô hình máy học như Random Forest, Extreme Gradient Boosting, Histogrambased Gradient Boosting, Multilayer Perceptron, Extra Trees, Adaptive Boosting.
- Một số tập dữ liệu Apache.

2. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu các vấn đề sau:

- Các mô hình học máy và học sâu áp dụng cho bài toán dự đoán lỗi phần mềm
- Nghiên cứu các mô hình oversampling và undersampling áp dụng trong việc xử lý mất cân bằng dữ liệu
- Nghiên cứu các mô hình mạng GAN để cân bằng dữ liệu
- Xây dựng thực nghiệm để áp dụng các mạng GAN trong cân bằng dữ liệu cho các mô hình dự đoán lỗi phần mềm

V. NỘI DUNG NGHIÊN CỨU

- Nghiên cứu đặc trưng phần mềm (metrics), vấn đề mất bằng dữ liệu trong dự đoán lỗi phần mềm.
- Nghiên cứu các mô hình máy học trong dự đoán lỗi phần mềm.
- Nghiên cứu các kỹ thuật cân bằng dữ liệu như SMOTE, ADASYN, Border-SMOTE, RUS,...
- Nghiên cứu và áp dụng mô hình GANs và các biến thể để đưa vào cân bằng dữ liệu nhằm nâng cao hiệu suất của mô hình dự đoán lỗi.
- Tiến hành thực nghiệm với các mô hình của GANs cân bằng dữ liệu trên các tập dữ liệu trong kho dữ liệu Promise để đánh giá kết quả đạt được và so sánh các kỹ thuật cân bằng dữ liệu truyền thống.
- Viết báo cáo tổng kết.

Danh sách bảng

1	Kết quả mô hình dự đoán lỗi phần mềm không áp dụng kỹ thuật lấy mẫu và lựa chọn đặc trưng	21
2	Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Chi-Squared and GAN.	22
3	Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Information Gain và GAN.	25
4	Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Fisher và GAN.	26
5	Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Relief và GAN.	27
6	Kết quả so sánh của kết hợp các kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và kỹ thuật lấy mẫu VanillaGAN.	28
7	Kết quả so sánh của kết hợp các kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và kỹ thuật lấy mẫu VanillaGAN (tt).	29

1 Tổng quan đề tài

Một lỗi trong hệ thống phần mềm được mô tả là một sai sót cấu trúc có thể gây ra hệ thống bị lỗi trong tương lai. Dự đoán lỗi phần mềm là một hoạt động quan trọng và cần thiết để tăng chất lượng phần mềm và giảm thiểu nỗ lực bảo trì trong các giai đoạn ban đầu của phát triển phần mềm. Dự đoán lỗi sớm có thể dẫn đến việc giải quyết kịp thời những lỗi này và cung cấp phần mềm có khả năng bảo trì tốt. Dự đoán lỗi phần mềm giúp cải thiện chất lượng phần mềm, trong đó các lỗi được dự đoán dựa trên kiến thức trước đó dưới dạng các tập dữ liệu. Có các kỹ thuật Khai Phá Dữ liệu (Data Mining), Máy Học (Machine Learning) và Học Sâu (Deep Learning) mà chúng ta sử dụng để dự đoán lỗi. Những kỹ thuật này được sử dụng để xây dựng các mô hình có thể dùng để dự đoán các lớp có lỗi và không có lỗi.

Dự đoán Lỗi Phần mềm (Software Fault Prediction - SFP) được thực hiện bằng cách phân chia các mô-đun/lớp thành các loại dễ lỗi và không lỗi. Chúng ta sử dụng các tập dữ liệu lỗi từ các dự án tương tự hoặc các bản cập nhật trước của các hệ thống phần mềm. Sau đó, chúng ta áp dụng các mô hình đã phát triển vào các mô-đun, lớp hoặc phương thức của các dự án phần mềm hiện tại, và phân loại chúng là mô-đun, lớp hoặc phương thức dễ lỗi hoặc không lỗi/không có lỗi trong hệ thống đang được kiểm tra. Chúng ta cho phép các chuyên gia kiểm thử tập trung thời gian và tài nguyên của họ vào các khu vực có vấn đề của hệ thống đang phát triển bằng cách sử dụng dự đoán lỗi phần mềm. Chúng ta có thể tận dụng các phương pháp dự đoán lỗi và hướng nỗ lực và nguồn lực của mình đến những nơi mà chúng ta dự đoán có lỗi. Một lợi ích khác của SFP là nhận thức về phân bố lỗi trước khi tiến hành kiểm tra, khi chúng ta sử dụng dữ liệu trước đó để cải thiện chất lượng của các bản phát hành tiếp theo vì chúng ta có ý tưởng về sự hiện diện của lỗi trong một phần cụ thể của hệ thống kiểm thử trước khi bắt đầu kiểm tra.

Mục tiêu của các mô hình dự đoán lỗi phần mềm là xác định các mô-đun dễ bị lỗi (ví dụ: tập tin, lớp, gói và chức năng) trong hệ thống phần mềm nhất định [70].

Những mô hình dự đoán này giúp người lập trình tập trung vào các mô-đun phần mềm dễ bị lỗi và phân bổ nguồn lực của họ một cách tối ưu nhất [40]. Kỹ thuật học máy chủ yếu đã được áp dụng trong nhiều năm để phát triển các mô hình dự đoán lỗi phần mềm. Các kỹ thuật này sử dụng dữ liệu lỗi lịch sử của phần mềm (thu được từ các dự án phần mềm trước đó) để huấn luyện các mô hình dự đoán và dự đoán các mô-đun bị lỗi [69, 53]. Dữ liệu lỗi lịch sử chứa các giá trị của các độ đo phần mềm khác nhau (ví dụ: coupling between objects, lines of code) và các nhãn (lỗi - không lỗi) cho mỗi mô-đun trong tập dữ liệu. Trong quá trình huấn luyện, mô hình dự đoán thu được kiến thức về đặc điểm của dự án phần mềm và sử dụng kiến thức này để dự đoán xem mô-đun khác có bị lỗi hay không [32].

Các mô hình SFP bị ảnh hưởng bởi một loạt các yếu tố. Những yếu tố này bao gồm vấn đề overfitting của mô hình, nhiễu trong tập dữ liệu, các tham số chi phí, các độ phần mềm, lựa chọn đặc trưng, v.v. Một trong những vấn đề nghiêm trọng nhất với các tập dữ liệu lỗi là vấn đề mất cân bằng dữ liệu. Một tập dữ liệu thiên lệch với sự phân bố không đều của các lớp được gọi là tập dữ liệu mất cân bằng. Một lớp dữ liệu chiếm đa số, trong khi lớp dữ liệu khác chiếm thiểu số. Khi một mô hình được áp dụng cho tập dữ liệu này, nó tạo ra kết quả thiên lệch, dẫn đến đánh giá hệ thống không chính xác. Để giải quyết vấn đề này, các thuật toán lấy mẫu khác nhau được sử dụng. Những thuật toán lấy mẫu này được kết hợp với các thuật toán học mẫu để tạo ra các mô hình SFP hiệu suất cao.

Tuy nhiên, hầu hết các bộ dữ liệu lỗi lịch sử bao gồm nhiều mô-đun không bị lỗi (tức là lớp đa số) hơn các mô-đun bị lỗi (tức là lớp thiểu số). Đây được gọi là vấn đề mất cân bằng lớp trong dự đoán lỗi phần mềm. Nhiều kỹ thuật học máy giả định rằng tỷ lệ mô-đun lớp thiểu số và mô-đun lớp đa số là xấp xỉ bằng nhau [4]. Do đó, các mô hình dự đoán được xây dựng trên bộ dữ liệu phần mềm không cân bằng có khả năng không học được các mẫu mô-đun bị lỗi và do đó có xu hướng dự đoán không chính xác nhiều mô-đun bị lỗi [44]. Điều này có thể dẫn đến các mô hình dự đoán không đáng tin cậy và không thể sử dụng trong thực tế. Do đó, việc đề xuất các kỹ thuật/phương pháp để giải quyết vấn đề mất cân bằng lớp của bộ dữ liệu lỗi là một vấn đề được cân nhắc kỹ lưỡng trong dự đoán lỗi phần mềm.

Để giải quyết vấn đề này, các thuật toán lấy mẫu khác nhau được sử dụng. Những thuật toán lấy mẫu này được kết hợp với các thuật toán học mẫu để tạo ra các mô hình SFP hiệu suất cao. Các kỹ thuật lấy mẫu dữ liệu như oversampling và under-sampling được sử dụng rộng rãi để giải quyết vấn đề mất cân bằng lớp trong dự đoán lỗi phần mềm [44, 6]. Kỹ thuật oversampling tăng số lượng mẫu ở lớp thiểu số. Ngược lại, kỹ thuật under-sampling loại bỏ một số mô-đun lớp đa số để cân bằng tỷ lệ mô-đun lớp thiểu số và lớp đa số [39]. Cả hai kỹ thuật đều nhằm mục đích cân bằng việc biểu diễn các mô-đun lớp thiểu số và lớp đa số để cải thiện hiệu suất của mô hình dự đoán. Trong dự đoán lỗi phần mềm, các kỹ thuật oversampling thường được sử dụng so với các kỹ thuật under-sampling, vì kỹ thuật lấy mẫu dưới đây sẽ loại bỏ các mô-đun một cách tùy ý khỏi tập dữ liệu. Do đó, một số thông tin quan trọng có thể bị mất khỏi tập dữ liệu. Để thực hiện kỹ thuật oversampling, các kỹ thuật như SMOTE và biến thể (ví dụ: ADASYN, Border-line SMOTE) và lấy mẫu (Random Oversampling (ROS)) hầu hết được sử dụng. Kỹ thuật ROS chỉ đơn giản sao chép các mô-đun lớp thiểu số để cân bằng tập dữ liệu mà không cần thêm bất kỳ thông tin mới nào [30]. Điều này dẫn đến một mô hình dự đoán sẽ gặp vấn đề overfitting - mô hình sẽ đúng trên tập huấn luyện nhưng trên tập test thì cho ra kết quả không chính xác, vì vậy có thể không hoạt động chính xác đối với các mô-đun thử nghiệm được. Các kỹ thuật dựa trên SMOTE tạo ra các mô-đun lớp thiểu số tổng hợp bằng cách nội suy các mô-đun lân cận đã xác định (dựa trên khoảng cách) để cân bằng tập dữ liệu [41, 14]. Các mô-đun mới được thêm vào này không trùng lặp như trong ROS, nhưng do các mô-đun lân cận, các kỹ thuật dựa trên SMOTE không tạo ra các mô-đun lớp thiểu số đa dạng, điều này có thể dẫn đến việc overfitting trên các mô hình dự đoán (hiệu suất thấp trên tập hợp con thử nghiệm). Một số công trình khác như Feng et al. [56] và Bennin và Bennin [23, 24], đã đề xuất các kỹ thuật lấy mẫu quá mức dựa trên độ phức tạp (complexity-based) và dựa trên tính đa dạng (diversity-based) tương ứng. Tuy nhiên, những kỹ thuật này tạo ra hiệu suất dự đoán lỗi hạn chế. Các kỹ thuật cost-sensitive [22] và các kỹ thuật ensemble-based techniques [55] cũng đã được sử dụng để học về sự mất cân bằng trong dự đoán lỗi phần mềm. Tuy

nhien, Galar [37] đã phát hiện ra rằng các kỹ thuật học tập bagging và boosting sẽ cải thiện độ chính xác của các mô hình dự đoán lỗi phần mềm nhưng không được thiết kế để xử lý các tập dữ liệu không cân bằng.

Generative Adversarial Networks (GAN) [2] gần đây đã được áp dụng thành công để giải quyết các vấn đề mất cân bằng dữ liệu trong một số lĩnh vực, chẳng hạn như tạo hình ảnh và video và xử lý ngôn ngữ tự nhiên. GAN chủ yếu được phát triển để tạo hình ảnh, dạng bảng và các loại dữ liệu khác [26, 54]. Các phương pháp tiếp cận GAN đã được chứng minh là có thể tìm hiểu cách phân phối tốt hơn các kỹ thuật lấy mẫu dữ liệu cơ sở và khắc phục một cách hiệu quả vấn đề về các lớp không cân bằng [54]. Trong nghiên cứu này, chúng tôi tập trung vào đánh giá thực nghiệm các kỹ thuật GAN tiềm năng và các biến thể của nó để tạo ra bộ dữ liệu lỗi phần mềm trong bài toán dự đoán lỗi. Ngoài ra, các đặc trưng không liên quan và dư thừa cũng ảnh hưởng đến tốc độ và độ chính xác của các mô hình học máy được đào tạo [68]. Kỹ thuật lựa chọn đặc trưng phần mềm là cần thiết để chọn các tập hợp con tốt nhất của độ đo phần mềm nhằm đạt được kết quả dự đoán tốt [10]. Do đó, trong nghiên cứu này, chúng tôi đã áp dụng bốn kỹ thuật lựa chọn đặc trưng để chọn ra các độ đo/đặc trưng phần mềm tối ưu có hiệu quả để huấn luyện các mô hình dự đoán lỗi. Ngoài ra, mục tiêu chính trong nghiên cứu của chúng tôi là xử lý vấn đề mất cân bằng lớp có thể cải thiện hiệu suất dự đoán. Để khắc phục cả tình trạng mất cân bằng lớp và loại bỏ đặc trưng không liên quan/dư thừa, chúng tôi áp dụng ba mô hình GAN bao gồm VanillaGAN, CTGAN và WANGGP [54] để tạo các mẫu tổng hợp nhằm cân bằng giữa nhóm thiểu số và nhóm đa số trong bộ dữ liệu lỗi phần mềm kết hợp bốn kỹ thuật lựa chọn đặc trưng (Chi-Squared, Information Gain, Fisher và Relief) dựa trên xếp hạng để xác định sự kết hợp hoạt động tốt nhất cho các mô hình dự đoán lỗi. Chúng tôi đã xem xét đánh giá mô hình dự đoán lỗi trên các độ đo đánh giá bao gồm Precision, Recall, F1-score và Area Under the ROC Curve (AUC) để so sánh hiệu suất của các kết hợp khác nhau giữa kỹ thuật lựa chọn đặc trưng và các phương pháp xử lý dữ liệu mất cân bằng GAN. Đóng góp của báo cáo này như sau:

1. Tiến hành thực nghiệm để kiểm tra cách tiếp cận kết hợp giữa 4 kỹ thuật lựa chọn đặc trưng dựa trên bộ lọc và 3 kỹ thuật oversampling trên 4 bộ dữ liệu lỗi được trích xuất từ kho lưu trữ PROMISE [71] sử dụng các thuật toán phân loại khác nhau như Random Forest (RF), Extra Tree (ET), AdaBoost (AdaBoost), Histogram-based Gradient Boosting (HGB) để đưa ra kết luận về các kết hợp tốt nhất.
2. Hiệu suất của các mô hình dự đoán lỗi được so sánh với việc áp dụng các kỹ thuật lấy mẫu và lựa chọn đặc trưng để kiểm tra tác động đáng kể của các kỹ thuật này trong việc xử lý các đặc trưng không liên quan/dư thừa và giải quyết hiệu quả vấn đề mất cân bằng lớp.

Phần còn lại của bài báo như sau: Mục 2 cung cấp kiến thức nền tảng về dự đoán lỗi phần mềm và các bước liên quan trong quá trình này. Các nghiên cứu liên quan được trình bày trong mục 3 của báo cáo. Mục 4 trình bày các mô hình GAN cho việc lấy mẫu dữ liệu trong dự đoán lỗi phần mềm. Xây dựng thực nghiệm và kết quả thực nghiệm được trình bày trong mục 5. Mục 6 sẽ trình bày các kết luận của báo cáo.

2 Dự đoán lỗi phần mềm

2.1 Tổng quan về dự đoán lỗi phần mềm

Dự đoán lỗi phần mềm là kỹ thuật dự đoán các đoạn phần mềm có lỗi trước giai đoạn kiểm thử trong quá trình phát triển phần mềm. Điều này tạo điều kiện sử dụng tài nguyên hiệu quả và hoàn thành phát triển phần mềm đúng hạn. Để xây dựng các mô hình dự đoán lỗi, các thuật toán học máy khác nhau được áp dụng. Các mô hình ensemble models là những mô hình được áp dụng nhiều nhất gần đây cho dự đoán lỗi. Hiệu suất của quá trình SFP bị ảnh hưởng bởi nhiều yếu tố. Tất cả những yếu tố này đóng vai trò quan trọng trong việc xây dựng một mô hình có cấu trúc và chính xác. Một số yếu tố bao gồm:

1. Độ đo phần mềm

2. Tập dữ liệu
3. Các kỹ thuật học máy
4. Lựa chọn các đặc trưng phần mềm
5. Độ đo đánh giá
6. Các vấn đề về tập dữ liệu

Trong khi phát triển một mô hình để phát hiện lỗi trong một mô-đun phần mềm, các nhà phát triển phải đối mặt với nhiều thách thức. Những yếu tố này có thể ảnh hưởng đến hiệu quả và độ chính xác của mô hình. Các nhà phát triển cần chú ý đến tất cả những yếu tố này khi phát triển một công cụ. Một số yếu tố bao gồm:

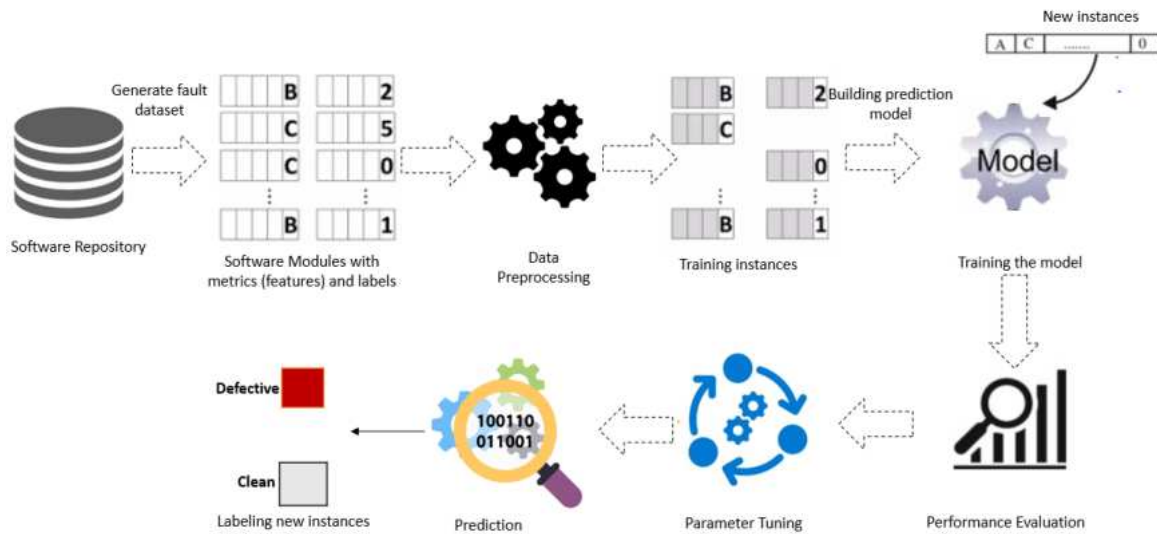
1. Mất cân bằng tập dữ liệu
2. Tập dữ liệu bị nhiễu
3. Vấn đề các mô hình huấn luyện bị overfitting
4. Các tham số mô hình

Kỹ thuật lấy mẫu quá mức (oversampling) và lấy mẫu thiếu (under-sampling) thường được sử dụng để cân bằng dữ liệu nhằm giải quyết các vấn đề mất cân bằng lớp. Trong báo cáo này, chúng tôi tập trung đến vấn đề mất cân bằng dữ liệu và các kỹ thuật lấy mẫu liên quan.

2.2 Quy trình dự đoán lỗi phần mềm

Dự đoán lỗi phần mềm là một phương pháp dự đoán các mô-đun dễ bị lỗi của các dự án phần mềm trước giai đoạn thử nghiệm. Dự đoán lỗi phần mềm tạo điều kiện thuận lợi cho việc chẩn đoán sớm và xác định các thành phần có thể chứa lỗi và do đó giúp các nhà phát triển phần mềm phát hiện hầu hết các lỗi xảy ra trong bản phát hành phần mềm hiện tại. Mô hình dự đoán lỗi phần mềm được đào tạo dựa trên dữ liệu lỗi lịch sử bằng cách sử dụng mô hình học máy từ các phiên bản phần mềm trước đó. Sau đó, mô hình được đào tạo sẽ được áp dụng để dự đoán

lỗi của các mô-đun trong phiên bản hiện tại của dự án. Như được minh họa trong Hình. 1, quy trình dự đoán lỗi phần mềm bao gồm các bước sau:



Hình 1: Quy trình dự đoán lỗi phần mềm.

1. **Thu thập dữ liệu:** Thu thập tập dữ liệu lỗi được tạo hoặc có sẵn công khai từ nhiều kho phần mềm khác nhau như PROMISE, NASA và Apache.
2. **Tiền xử lý dữ liệu:** Tập dữ liệu lỗi phần mềm có thể chứa nhiều, dữ liệu nhiều chiều, sự hiện diện của các thuộc tính không liên quan hoặc dư thừa, các ngoại lệ và các lớp không cân bằng. Do đó, các kỹ thuật tiền xử lý dữ liệu khác nhau được áp dụng, chẳng hạn như chuẩn hóa dữ liệu, lựa chọn đặc trưng, trích xuất đặc trưng và lấy mẫu dữ liệu, để cải thiện hiệu suất của các mô hình dự đoán lỗi.
3. **Trích xuất các đặc trưng và xây dựng tập dữ liệu đào tạo:** Ở bước này, các độ đo phần mềm được trích xuất từ mã nguồn hoặc nhật ký lịch sử của các dự án phần mềm. Các độ đo phần mềm được trích xuất được kết hợp với thông tin lỗi để tạo tập dữ liệu huấn luyện được sử dụng làm đầu vào cho mô hình học máy để dự đoán lỗi.
4. **Xây dựng mô hình dự đoán lỗi:** Các kỹ thuật thống kê, mô hình học máy hoặc mô hình học sâu được khai thác để xây dựng mô hình dự đoán bằng

cách sử dụng tập dữ liệu huấn luyện. Mô hình được đào tạo sau đó được áp dụng để dự đoán lỗi trong các mô-đun dễ bị lỗi phần mềm.

5. **Đánh giá mô hình:** Sau các bước huấn luyện, tập dữ liệu thử nghiệm được sử dụng để kiểm tra mô hình dự đoán lỗi. Hiệu suất của mô hình có thể được đo bằng cách so sánh giá trị dự đoán của các mô-đun dễ bị lỗi do mô hình tạo ra với giá trị thực tế của các mô-đun dễ bị lỗi bằng cách sử dụng các biện pháp đánh giá như accuracy, precision, recall, the F1-score and AUC.
6. **Parameter Tuning:** Các hyperparameters của mô hình được điều chỉnh để đạt được độ chính xác cao và hiệu suất tốt hơn.
7. **Dự đoán lỗi:** Mô hình dự đoán lỗi được áp dụng để xác định các module phần mềm dễ bị lỗi trong các dự án thực tế.

2.3 Độ đo phần mềm

Để nắm bắt được các khía cạnh khác nhau của sản phẩm phần mềm, các nghiên cứu gần đây đã khai thác các độ đo để dự đoán lỗi phần mềm. Các thước đo chất lượng khác nhau được sử dụng để đo lường các đặc tính chất lượng của phần mềm trong công nghệ phần mềm. Chúng được phân loại thành bốn loại [52]: *procedural metrics* - độ đo thủ tục, *object-oriented metrics* - độ đo hướng đối tượng, *hybrid metrics* - độ đo kết hợp và *miscellaneous metrics*. độ đo thủ tục bao gồm các độ đo mã tĩnh ban đầu được trình bày bởi Halsted [33], McCabe [59] và độ đo LOC. Độ đo $v(g)$ của McCabe đo lường độ phức tạp của mô-đun phần mềm. độ đo hướng đối tượng, ban đầu được phát triển bởi Chidamber và Kemerer [51] và Lorenz và Kidd [31], đo lường các đặc điểm của hệ thống hướng đối tượng chẳng hạn như object classes, inheritance and cohesion. độ đo kết hợp bao gồm các độ đo hướng đối tượng và thủ tục trong dự đoán lỗi phần mềm. Các độ đo khác bao gồm các độ đo quy trình, độ đo thay đổi và độ đo churn. Một số nhà nghiên cứu đã đề xuất các mô hình dự đoán sử dụng cả độ đo phần mềm và một số kỹ thuật học máy. Theo Zimmermann *et al* [61], các độ đo phức tạp có thể được tổng hợp để dự đoán lỗi. Họ đã phát hành tập dữ liệu Eclipse, được tạo bằng cách ánh xạ các lỗi từ cơ

sở dữ liệu lỗi tới vị trí của chúng trong mã nguồn và sau đó công khai tất cả dữ liệu trên Kho lưu trữ máy học của UCI. Cata và Diri [5] đã xem xét các tài liệu kỹ yếu hội nghị và tạp chí được xuất bản trong lĩnh vực dự đoán lỗi phần mềm với nhiều loại tập dữ liệu, độ đo và phương pháp khác nhau. Họ kết luận rằng, mặc dù các độ đo ở cấp độ phương pháp được sử dụng rộng rãi trong nghiên cứu dự đoán lỗi trước đây, nhưng tỷ lệ sử dụng các độ đo ở cấp độ lớp, cấp thành phần và cấp quy trình vẫn vượt quá mức chấp nhận được của các mô hình dự đoán.

Ruchika và Malhotra [49] đã tiến hành đánh giá có hệ thống về nhiều loại độ đo phần mềm, kỹ thuật máy học và bộ dữ liệu. Họ kết luận rằng các thước đo hướng đối tượng là thước đo hiệu quả nhất để dự đoán lỗi phần mềm do mức độ lỗi thiết kế trong phần mềm cao. Cụ thể, LOC (dòng mã), CBO (khớp nối giữa các đối tượng) và RFC (phản hồi cho một lớp) là các độ đo hiệu quả cho các kỹ thuật lựa chọn đặc trưng. Ngược lại, độ đo DIT (độ sâu của cây kế thừa) và NOC (số lượng con) được đề cập là không hữu ích trong dự đoán lỗi phần mềm. Meiliana và Karim [34] đã tiến hành đánh giá tài liệu về một số độ đo phần mềm để dự đoán lỗi phần mềm bằng kỹ thuật máy học. Dựa trên quan sát của họ từ nhiều tài liệu nghiên cứu khác nhau, các độ đo cấp độ lớp đã được chứng minh là hiệu quả hơn các độ đo cấp phương pháp trong việc dự đoán lỗi. Nam và Kim [19] đã trình bày hai phương pháp dự đoán lỗi là CLA và CLAMI, sử dụng độ lớn của giá trị độ đo để gắn nhãn cho tập dữ liệu không được gắn nhãn. Họ kết luận rằng phương pháp của họ đạt được hiệu suất tốt hơn so với các mô hình dự đoán điển hình và có ứng dụng tiềm năng để dự đoán lỗi đối với các dự án có bộ dữ liệu không được gắn nhãn. Yang *et al.* [66] đã đề xuất một mô hình không giám sát bằng cách sử dụng các độ đo thay đổi để dự đoán lỗi đúng lúc, nhận thức được nỗ lực. Nam *et al.* [20] đã đề xuất một kỹ thuật dự đoán lỗi không đồng nhất bằng cách sử dụng các bộ độ đo không đồng nhất để dự đoán lỗi trong dự đoán lỗi liên dự án. Cách tiếp cận của họ mang lại kết quả đầy hứa hẹn và chứng tỏ mức độ khả thi cao.

2.4 Lựa chọn đặc trưng

Các thuật toán học máy thường được sử dụng cho nhiều ứng dụng khác nhau, bao gồm nhận dạng mẫu, khai thác dữ liệu và mô hình dự đoán. Các thuật toán này có khả năng học hỏi mạnh mẽ từ các không gian đặc trưng có nhiều chiều và nhiều thông tin. Tuy nhiên, không gian đặc trưng nhiều chiều có thể dẫn đến hiện tượng khớp quá mức, tăng độ phức tạp tính toán và giảm khả năng diễn giải mô hình. Lựa chọn đặc trưng [16] là một bước tiền xử lý quan trọng nhằm mục đích giảm kích thước của tập dữ liệu bằng cách chọn một tập hợp con các đặc trưng có liên quan từ một nhóm lớn các thuộc tính ứng cử viên. Phần này cung cấp cái nhìn tổng quan toàn diện về các kỹ thuật lựa chọn đặc trưng, bao gồm các phương thức filter, wrapper và embedded.

2.4.1 Lựa chọn đặc trưng dựa trên Filter

Các phương pháp lựa chọn đặc trưng dựa trên filter [42] dựa trên các đặc điểm xếp hạng bằng cách sử dụng các phép đo thống kê, chẳng hạn như mutual information, chi-squared, or correlation coefficients. Các phương thức này độc lập với mô hình phân lớp được sử dụng và các đặc trưng được chọn sẽ được sử dụng làm đầu vào cho mô hình phân lớp. Các phương pháp filter có hiệu quả về mặt tính toán vì chúng không yêu cầu đào tạo mô hình học máy cho từng tập hợp con đối tượng.

Một trong những phép đo thống kê thường được sử dụng trong các phương pháp lọc là thông tin lẫn nhau [10], đo lường sự phụ thuộc giữa hai biến. Thông tin tương hỗ giữa một đối tượng

$$MI(x, y) = \sum_{i=1}^c \sum_{j=1}^d (p(x_i, y_j) \log(p(x_i, y_j) / (p(x_i)p(y_j)))) \quad (1)$$

trong đó c và d lần lượt là số giá trị duy nhất của x và y , và $p(x_i)$ và $p(y_j)$ là giá trị cận biên xác suất của x_i và y_j tương ứng. Giá trị thông tin lẫn nhau cao cho thấy mối quan hệ chặt chẽ giữa đối tượng địa lý và biến mục tiêu, làm cho các đặc điểm này phù hợp hơn để sử dụng trong vectơ đầu vào.

2.4.2 Lựa chọn đặc trưng dựa trên Wrapper

Các kỹ thuật lựa chọn đặc trưng dựa trên wrapper [47] xem xét sự tương tác giữa một tập hợp các đặc trưng và một mô hình phân loại. Ý tưởng cơ bản là đánh giá hiệu suất của mô hình phân loại trên các tập hợp con đối tượng khác nhau, chọn tập hợp con mang lại hiệu suất tốt nhất. Quá trình tìm kiếm có thể được thực hiện bằng cách sử dụng tìm kiếm toàn diện, tìm kiếm tham lam hoặc tìm kiếm ngẫu nhiên. Một cách tiếp cận phổ biến là sử dụng thuật toán tìm kiếm tham lam, chẳng hạn như backward elimination, trong đó các thuộc tính được loại bỏ từng đặc trưng một cho đến khi đạt được hiệu suất thỏa đáng.

Phương thức lựa chọn đặc trưng dựa trên wrapper có thể được xây dựng dưới dạng một bài toán tối ưu hóa, trong đó mục tiêu là tìm ra tập hợp con đặc trưng tốt nhất giúp tối đa hóa hiệu suất của bộ phân loại. Đặt F là tập hợp tất cả các đặc trưng, S là tập con của F , và $J(S)$ là hiệu suất của bộ phân loại trên tập con đặc trưng S . Phương pháp lựa chọn đặc trưng dựa trên wrapper có thể được xây dựng như sau:

$$S^* = \operatorname{argmax}_S (J(S)) \quad (2)$$

where S^* là tập đặc trưng dữ liệu tối ưu thu được.

2.4.3 Lựa chọn đặc trưng dựa trên Embedded

Các kỹ thuật lựa chọn đặc trưng dựa trên Embedded [3] là sự kết hợp giữa các phương thức filter và wrapper, trong đó việc lựa chọn đặc trưng được tích hợp vào thuật toán học tập được mô hình phân lớp sử dụng. Các phương pháp này hiệu quả hơn về mặt tính toán so với các phương pháp bao bọc, vì quá trình lựa chọn đặc trưng được thực hiện như một phần của quá trình đào tạo. Các phương pháp Regularization, chẳng hạn như Lasso (Least Absolute Shrinkage and Selection Operator) và Ridge Regression, là những ví dụ phổ biến của các kỹ thuật lựa chọn đặc trưng dựa trên Embedded. Các phương pháp này sử dụng số hạng phạt để thu nhỏ hệ số của các đặc tính ít quan trọng hơn về 0, loại bỏ chúng khỏi mô hình một cách hiệu quả. Lasso [45], Hồi quy Ridge [11] và ElasticNet [11] là

các kỹ thuật chính quy hóa được sử dụng trong hồi quy tuyến tính để ngăn chặn quá mức và cải thiện độ chính xác dự đoán của mô hình.

2.5 Các mô hình học máy

Một số nghiên cứu [63, 35, 13] đã chứng minh rằng máy học (Machine Learning) có thể được áp dụng thành công để xây dựng các mô hình dự đoán lỗi phần mềm nhằm phân loại các nhãn Dễ xảy ra lỗi (Fault Prone) và Không dễ xảy ra lỗi (None-Fault-Prone). Nhiều nhà nghiên cứu đã phát hiện ra rằng hiệu suất của các mô hình dự đoán có sự khác biệt đáng kể đối với các mô hình phân loại khác nhau [25]. Các nghiên cứu của Malhotra [49] và Wang [9] đã được xem xét để lựa chọn các cấu hình và kỹ thuật máy học dùng trong nghiên cứu này. Trong số các kỹ thuật khác nhau được trình bày trong miền dự đoán lỗi phần mềm như học tập tổng hợp, mạng nơ-ron nhân tạo và mô hình học sâu, chúng tôi đã xem xét các kỹ thuật sau: Random Forest [12], Extreme Gradient Boosting (XGBoost) [62], Histogram-based Gradient Boosting (HGBBoost) [1], Multilayer Perceptron (MLP) [57], Extra Trees [43] và Adaptive Boosting (AdaBoost) [67].

Tumar *et al.* [18] đã đề xuất một mô hình SFP dựa trên tối ưu hóa Binary Moth Flame kết hợp với phương pháp lấy mẫu tổng hợp thích ứng (Adaptive Synthetic Sampling - ADASYN) để giải quyết vấn đề mất cân bằng dữ liệu. Khi áp dụng vào tập dữ liệu PROMISE, kỹ thuật được đề xuất này cải thiện hiệu suất của các bộ phân loại khác nhau. Trong số đó, Phân tích Phân biệt Tuyến tính (Linear Discriminant Analysis - LDA) có giá trị AUC cao nhất và KNN là tốt nhất về thời gian thực thi. Rathore *et al.* [54] đã giới thiệu ba phương pháp tạo mẫu quá mức riêng biệt là Conditional GAN (CT-GAN), Vanilla GAN, và Wasserstein GAN với Gradient Penalty (WGANGP). Thực nghiệm nghiệm được thực hiện trên các tập dữ liệu PROMISE, JIRA và Eclipse. Khi các phương pháp lấy mẫu này được sử dụng với các mô hình cơ sở trong các bài kiểm tra trên các tập dữ liệu lỗi, kết quả của các mô hình cơ sở được cải thiện đáng kể. Đối với SFP trong các mô hình trong phiên bản và giữa các phiên bản, Singh và Rathore [46] đã áp dụng các phương pháp hồi quy Bayesian phi tuyến tính và tuyến tính. Các phương pháp lấy

mẫu dữ liệu SMOTE được áp dụng cùng với Random Forest (RF), Support Vector Machine (SVM), Linear Regression (Lr), Linear Bayesian Regression (LBr), and Non-linear Bayesian Regression (NLBr). Hồi quy Bayesian phi tuyến tính đã vượt trội hơn các phương pháp hồi quy tuyến tính trên một mẫu chứa 46 dự án phần mềm khác nhau.

3 Các nghiên cứu liên quan

Dự đoán lỗi phần mềm là lĩnh vực phổ biến nhất trong công nghệ phần mềm miền trong ba thập kỷ qua [49]. Nhiều mô hình máy học đã được áp dụng trước đó cho các mô hình dự đoán lỗi như Decision tree, Naïve Bayes, Logistic Regression, Support Vector Machine, Multilayer Perceptron, Random Forest và Ensemble learning. Những kỹ thuật này mang lại hiệu suất đáng kể cho các mô hình dự đoán lỗi phần mềm. Tuy nhiên, trong bộ dữ liệu lỗi phần mềm, việc biểu diễn các mô-đun bị lỗi (lớp thiểu số) ít hơn nhiều so với các mô-đun không bị lỗi (lớp đa số), đây là vấn đề mất cân bằng lớp. Do vấn đề này, hiệu suất của các mô hình dự đoán lỗi bị giảm nghiêm trọng. Nhiều nghiên cứu đã cố gắng giải quyết vấn đề này của mô hình dự đoán lỗi phần mềm [23, 55]. Về cơ bản, các phương pháp lấy mẫu sampling, phương pháp ensemble và phương pháp cost-sensitive đã được áp dụng trước đó để giải quyết vấn đề. Các phương pháp lấy mẫu bao gồm kỹ thuật under-sampling và oversampling. Các kỹ thuật oversampling như SMOTE và ROS và các kỹ thuật under-sampling như RUS đã được áp dụng rộng rãi cho dự đoán lỗi phần mềm. Kamei [65] đã đánh giá các phương pháp lấy mẫu khác nhau (ROS, SMOTE, RUS và one-sided selection) cho dự đoán lỗi bằng các kỹ thuật học máy khác nhau trong công việc của họ. Các tác giả đã chỉ ra rằng các phương pháp lấy mẫu giúp cải thiện hiệu suất của các mô hình tuyến tính và mô hình logistic, trong khi mạng nơ-ron và mô hình cây phân loại lại hoạt động kém hơn. Trong một nghiên cứu tương tự khác, Tan [38] đã đánh giá bốn phương pháp lấy mẫu (simple duplicate, spread subsample, SMOTE, and resampling with/without replacement). Kết quả thử nghiệm cho thấy phương pháp resampling hoạt động tốt nhất trong số tất cả

các phương pháp lấy mẫu tổng thể và cải thiện hiệu suất dự đoán.

Các phương pháp ensemble learning giải quyết vấn đề mất cân bằng lớp bằng cách tạo và kết hợp một số mô hình học máy để có được mô hình học máy mạnh. Các phương pháp tập hợp được sử dụng phổ biến nhất là bagging, boosting, and stacking. Một số biến thể như Under-Bagging, OverBagging, UnderOverBagging, SMOTBagging, RUSBoost và SMOTEBoost đã được áp dụng cho dự đoán lỗi phần mềm [44], [7]. Siers và Islam [36] đã đề xuất một phương pháp cost-sensitive và tiếp cận knowledge discovery cho việc học không cân bằng trong dự đoán lỗi phần mềm. Các tác giả đã xác nhận tính hiệu quả của phương pháp được đề xuất cho bộ dữ liệu của NASA và kết quả cho thấy phương pháp được trình bày vượt trội hơn các phương pháp lấy mẫu được so sánh khác. Gần đây, một số nghiên cứu như Malhotra và Kamal [48], Song [44], Phong [56], và Gong [29] đã khám phá việc sử dụng các phương pháp lấy mẫu và các phương pháp khác để xử lý vấn đề mất cân bằng lớp trong dự đoán lỗi phần mềm. Các công trình này báo cáo rằng việc giải quyết vấn đề mất cân bằng lớp trong việc xây dựng mô hình dự đoán lỗi sẽ mang lại hiệu suất được cải thiện tổng thể.

4 Mô hình GAN cho lấy mẫu dữ liệu trong dự đoán lỗi phần mềm

4.1 Mô hình mạng GAN

Mạng GAN được GoodFellow *et al.* [15] giới thiệu vào năm 2014 và được định nghĩa là một trò chơi giữa hai mạng nơ-ron nhân tạo thường được gọi là *generator* và *discriminator*. Bộ *generator* cố gắng tìm hiểu tỷ lệ phân phối của dữ liệu thực và tạo ra các phiên bản mới có thể đánh lừa bộ *discriminator*. Mục đích của bộ *discriminator* là phân loại các mẫu thực và tạo ra các mẫu chính xác nhất có thể. Cả bộ *generator* và *discriminator* đều được huấn luyện đồng thời và cũng chạy cạnh tranh với nhau trong giai đoạn huấn luyện. Mục tiêu của quá trình đào tạo GAN là tìm ra điểm cân bằng Nash [28], đây là trạng thái mà cả hai mạng đều có

thể đạt được hiệu suất tối ưu. Trạng thái cân bằng Nash trong GAN đạt được khi bộ *generator* tạo ra dữ liệu tổng hợp không thể phân biệt được với dữ liệu thực và bộ *discriminator* không thể cải thiện khả năng phân biệt giữa mẫu thực và mẫu được tạo. Tại thời điểm này, GAN đã nắm bắt được sự phân bố của dữ liệu thực và có thể tạo ra các mẫu dữ liệu mới tương tự với dữ liệu thực. Do ứng dụng thành công của GAN cho nhiều ứng dụng nghiên cứu và đã được khai thác để áp dụng trong nhiều lĩnh vực như xử lý ngôn ngữ tự nhiên, thị giác máy tính và mô hình phân loại. Van Sloun *et al.* [50] đã khai thác GAN để tạo ra hình ảnh có độ phân giải cao từ hình ảnh siêu âm thu được bằng cách tích hợp nhiều mô hình học sâu khác nhau. Fathi-Kazerooni [58] giới thiệu GAN Tunnel để phát hiện và xác định tình hình giao thông nhằm cải thiện chức năng điều khiển tự lái của ô tô.

4.2 VanillaGAN

Vanilla GAN [15] là phiên bản được xuất bản sớm nhất của Mạng GAN. Kiến trúc mô hình GAN bao gồm hai mô hình con: mô hình *generator* và mô hình *discriminator*. Với một số biến nhiễu z làm đầu vào, bộ G sẽ tạo ra các mẫu gần với dữ liệu mục tiêu thực hơn. Mô hình phân biệt D cố gắng phân biệt xem các mẫu đầu vào đến từ bộ tạo G hay từ dữ liệu thực. Đối tượng min-max có thể được hình thức hóa như sau:

$$\min_{(G)} \max_{(D)} (E_{x \sim p_D} [\log(D(x))] + E_{z \sim p_z} [\log(1 - D(G(z)))]) \quad (3)$$

trong đó, x là dữ liệu đầu vào, p_D và p_z lần lượt thể hiện sự phân bố của dữ liệu thực và dữ liệu được sinh ra, $z \sim p_z$ là nhiễu lấy mẫu từ phân phối Gauss or phân phối đồng nhất. Mục tiêu này dựa trên hàm loss Binary-Cross Entropy (BCE). Đầu ra của Discriminator D nằm trong phạm vi $[0,1]$ bằng cách sử dụng hàm kích hoạt sigmoid. Để tìm hiểu các tham số của GAN, bộ phân biệt D cần được đào tạo cho đến khi đạt được độ chính xác tối đa để phân biệt dữ liệu đầu vào với dữ liệu được tạo $G(z)$ hoặc dữ liệu thực. Bộ *generator* được huấn luyện để đạt được mức tối thiểu $\log(1 - D(G(z)))$. Quá trình huấn luyện được thực hiện luân phiên

cho đến khi đạt được giải pháp chung trong trường hợp $p_{data} = p_g$.

4.3 Conditional GAN - CTGAN

CTGAN [27] là một biến thể của GAN để lập mô hình phân phối dữ liệu dạng bảng. Để xử lý các cột chứa phân phối không phải Gaussian và đa phương thức, CTGAN thay thế chuẩn hóa tối thiểu - tối đa bằng cách khai thác chuẩn hóa theo chế độ cụ thể. Trong chuẩn hóa theo chế độ cụ thể, các giá trị liên tục được chuyển đổi thành các vectơ giới hạn là đầu vào phù hợp với mạng nơ-ron. CTGAN áp dụng mô hình Hỗn hợp Gaussian (VGM) có thể thay đổi [8] để dự báo số lượng modes và điều chỉnh Gaussian Mixture cho mỗi cột giá trị liên tục. Gaussian Mixture đã học được định nghĩa như sau:

$$\mathbb{P}_{c_i}(c_{i,j}) = \sum_{k=1}^l \mu_k \mathbb{N}(c_{i,j}; \eta_k, \mathcal{O}_k) \quad (4)$$

trong đó μ_k là trọng số và $\mathcal{O}_k$ là độ lệch chuẩn của một chế độ, trong đó xác suất ρ_k của $c_{i,j}$ được tính toán cho từng mode. Việc tính toán mật độ xác suất được định nghĩa như sau:

$$\rho_k = \mu_k \mathbb{N}(c_{i,j}; \eta_k, \phi_k) \quad (5)$$

Sau đó, một mode được lấy mẫu từ mật độ xác suất đã cho và mode lấy mẫu được sử dụng để chuẩn hóa giá trị. $c_{i,j}$ được trình bày bằng cách sử dụng một one-hot vector $\beta_{i,j}$ có kích thước bằng số lượng các mode và giá trị vô hướng $\alpha_{i,j}$ được sử dụng để trình bày giá trị trong mode. Do đó, một hàng trong tập dữ liệu là sự nối của các cột liên tục và rời rạc:

$$r_j = \alpha_{1,j} \oplus \beta_{1,j} \oplus \dots \oplus \alpha_{N_c,j} \beta_{N_c,j} \oplus d_{1,j} \oplus \dots \oplus d_{N_d,j} \quad (6)$$

trong đó $d_{i,j}$ là một one-hot vector của các giá trị rời rạc.

Để giải quyết vấn đề mất cân bằng dữ liệu trong mô hình phân biệt đối xử, CTGAN triển khai một bộ *generator* có điều kiện và huấn luyện theo mẫu. Đặc biệt, dữ liệu được lấy mẫu lại một cách hiệu quả theo cách tất cả các loại biến rời rạc được lấy mẫu trong quá trình huấn luyện và dữ liệu thực sẽ được phục hồi trong giai đoạn thử nghiệm.

4.4 Wasserstein GAN With Gradient Penalty - WGANP

WGAN hay Wasserstein Progressive Growing of Generative Adversarial Networks là một mô hình tổng quát dựa trên mạng GAN, được Karras *et al.* [60] giới thiệu vào năm 2017. Bằng cách áp dụng hàm mục tiêu ổn định hơn, khoảng cách Wasserstein, làm nền tảng cho việc huấn luyện, mô hình này nhằm khắc phục những khó khăn về tính không ổn định và sụp đổ chế độ vốn có trong các GAN thông thường. Tuy nhiên, trong công thức WGAN ban đầu, độ dốc của hàm mất mát phê bình có thể gặp phải vấn đề vanishing gradient (mất mát đạo hàm, đạo hàm quá nhỏ, gần như bằng 0), trong đó độ dốc trở gradient nên rất nhỏ và gây ra sự hội tụ chậm hoặc thậm chí dừng quá trình huấn luyện. Do đó, bắt đầu với độ phân giải thấp và dần dần mở rộng kích thước đầu vào của mạng *generator*, mô hình WANGP được huấn luyện bằng cách tăng dần độ phân giải của hình ảnh được tạo ra.

Kiến trúc WANGP có thể được biểu diễn về mặt toán học dưới dạng trò chơi minimax hai người chơi giữa mạng Discriminator D và mạng Generator G , với mục tiêu là xác định Điểm cân bằng Nash giữa hai mạng. Sau đây là biểu thức của hàm mục tiêu để huấn luyện mô hình WANGP:

$$\min_G \max_D (\mathbb{E}_{x \sim p_D}[D(x)] - \mathbb{E}_{z \sim p_z}[D(G(z))]) - \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} [(|| \nabla_{\hat{x}} D(\hat{x}) ||_2 - 1)^2] \quad (7)$$

trong đó x là mẫu từ phân phối dữ liệu thực $p_{data}(x)$, z là vectơ nhiễu ngẫu nhiên, G là mạng Generator, D là mạng Discriminator và λ là hệ số phạt (penalty coefficient) [17].

5 Thực nghiệm đề xuất

Như đã đề cập trong phần giới thiệu, dự đoán lỗi phần mềm là một quy trình quan trọng trong công nghệ phần mềm và cũng phụ thuộc vào các bộ dữ liệu lịch sử đã được thu thập từ các dự án phần mềm trước đó. Và quá trình này vẫn còn hai thách thức lớn bao gồm dữ liệu vẫn còn chứa các thuộc tính dư thừa hoặc không liên quan đến lỗi và các lớp không cân bằng. Vì vậy chúng tôi đề xuất hai

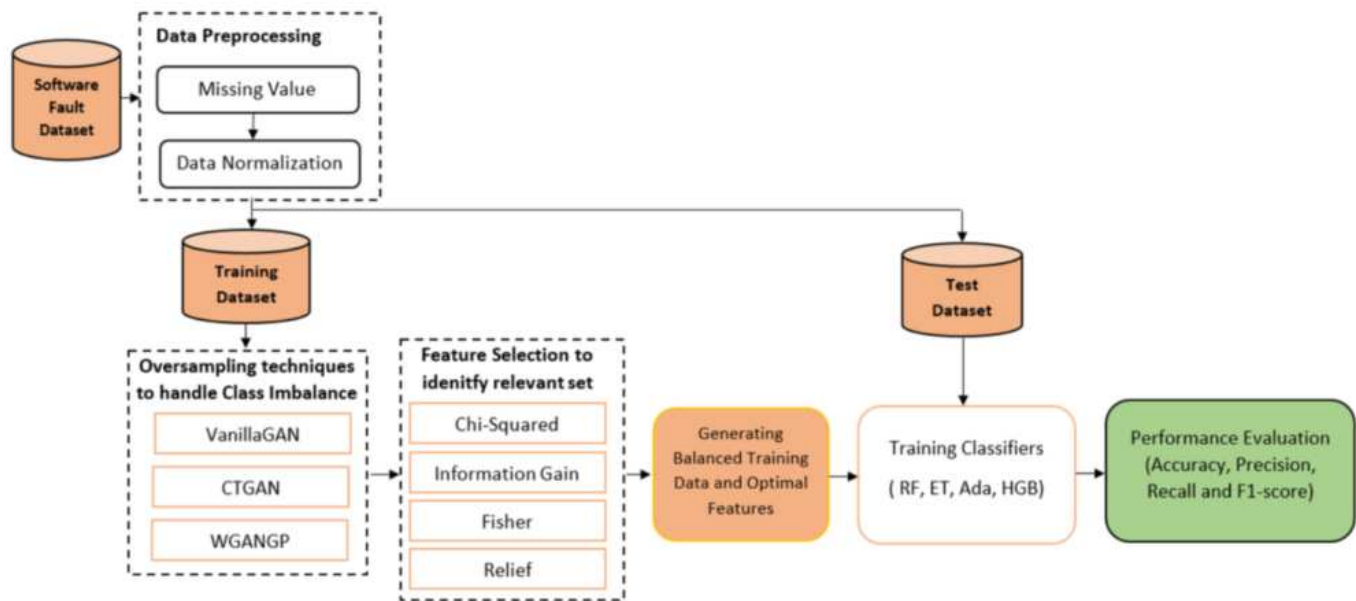
các phương pháp tiếp cận để kiểm tra việc xác định các kết hợp giữa các kỹ thuật lựa chọn đặc trưng và lấy mẫu dữ liệu nhằm mang lại hiệu quả hay không trong việc cải thiện hiệu suất và độ chính xác của mô hình dự đoán lỗi đã phát triển.

5.1 Thực nghiệm dựa trên kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Filter (Filter-based feature selection) và các kỹ thuật lấy mẫu GAN

Các bước của phương pháp thực nghiệm đề xuất cho việc kết hợp lựa chọn đặc trưng dựa trên Filter (filter-based feature selection) và các kỹ thuật lấy mẫu dữ liệu GAN được hiển thị trong Hình 2. Đầu tiên, chúng tôi đã thu thập bốn bộ dữ liệu lỗi là CM1, KC1, KC2 và PC1 từ PROMISE kho. Chuẩn hóa dữ liệu bằng kỹ thuật chuẩn hóa z được áp dụng cho giai đoạn tiền xử lý. Các tập dữ liệu chuẩn hóa được chia thành tập huấn luyện và tập kiểm tra. Sau khi áp dụng ba mô hình lấy mẫu quá mức GAN bao gồm VanillaGAN, CTGAN và WANGGP để cân bằng các tập dữ liệu huấn luyện, bốn lựa chọn đặc trưng được lựa chọn dựa trên filtering - xếp hạng, cụ thể là Chi-Squared, Information Gain, Fisher và Relief để trích xuất $\log_2 N$ đặc trưng (N - tổng số độ đo phần mềm trong bộ dữ liệu lỗi đầy đủ). Các tập dữ liệu huấn luyện cân bằng có các đặc trưng tối ưu tiếp theo sẽ được huấn luyện trên Random Forest (RF), Extra Tree (ET), AdaBoost (AB) và HistGradientBoosting (HG). Các bộ thực nghiệm sau đó được đưa vào các mô hình dự đoán lỗi để so sánh hiệu suất của các mô hình này bằng cách sử dụng kết hợp các kỹ thuật chọn đặc trưng và lấy mẫu dữ liệu trên các độ đo đánh giá bao gồm Precision, Recall, F1-score and AUC.

5.2 Thực nghiệm dựa trên kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Wrapper (wrapper-based feature selection) và các kỹ thuật lấy mẫu GAN

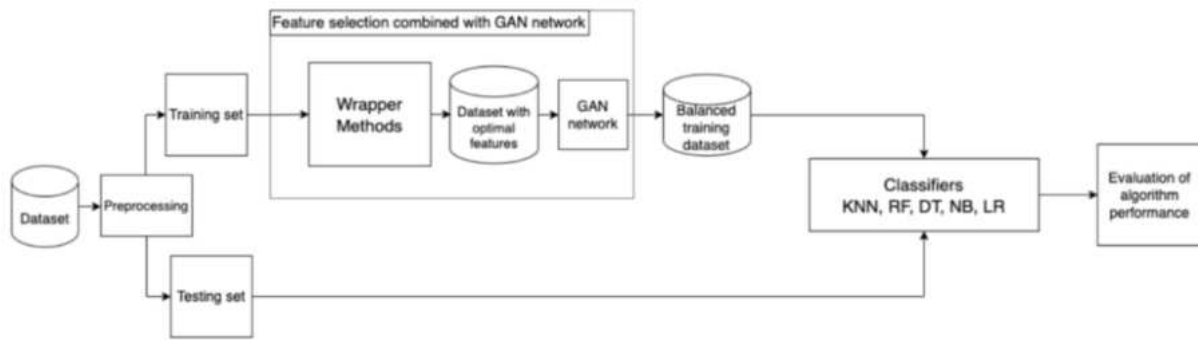
Phương pháp lựa chọn đặc trưng dựa trên wrapper sử dụng thuật toán học máy để đánh giá hiệu suất của một tập hợp con của các đặc trưng. Thuật toán được



Hình 2: Mô hình kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Filter và các kỹ thuật lấy mẫu GAN.

huấn luyện bằng cách sử dụng các tập hợp con đặc trưng khác nhau và tập hợp con mang lại hiệu suất tốt nhất được chọn. Một thực nghiệm thứ 2 được thực nghiệm để kiểm tra tính hiệu quả của việc kết hợp sáu phương pháp Wrapper khác nhau cho chọn tập hợp con các đặc trưng tối ưu và VanillaGAN là một biến thể của mạng GAN để cân bằng tỷ lệ mô-đun bị lỗi và không bị lỗi trong bộ dữ liệu lỗi. Cách tiếp cận tổng thể được thể hiện trong Hình 3. Trong quá trình tiền xử lý dữ liệu giai đoạn này, chúng tôi điền các giá trị còn thiếu và áp dụng chuẩn hóa Z để chuẩn hóa dữ liệu. Tiếp theo, sáu phương pháp wrapper khác nhau, cụ thể là Genetic Algorithm(GA), Particle Swarm Optimization (PSO), Whale optimization algorithm (WOA), Cuckoo search (CS), Mayfly Algorithm(MA) và Binary Bat Algorithm (BBA) để chọn các đặc trưng quan trọng và phù hợp nhất để giảm bớt các thuộc tính dư thừa/ không liên quan đến lỗi. Các bộ dữ liệu với đặc trưng tối ưu được sử dụng để huấn luyện mạng VanillaGAN để khắc phục tình trạng mất cân bằng dữ liệu. Cuối cùng, các tập dữ liệu huấn luyện và tập dữ liệu thực nghiệm này được sử dụng để đào tạo bằng các kỹ thuật học máy, bao gồm: Neighbors (KNN), Random Forest (RF), Decision Tree (DT), Naïve Bayes (NB)

and Logistic Regression (LR).



Hình 3: Mô hình kết hợp giữa kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và các kỹ thuật lấy mẫu GAN.

5.3 Tập dữ liệu thực nghiệm

Năm bộ dữ liệu lỗi khác nhau từ kho lưu trữ Promise [21] đã được sử dụng trong nghiên cứu này. Những bộ dữ liệu này đã được sử dụng rộng rãi trong nhiều nghiên cứu về lỗi phần mềm nghiên cứu dự báo [64]. Như được hiển thị trong Bảng ??, các bộ dữ liệu không cân bằng. Các thuộc tính độc lập là các độ đo mã nguồn như Độ phức tạp theo chu kỳ của McCabe ($v(g)$), Độ sâu của cây kế thừa (dit), v.v. và các thuộc tính phụ thuộc bao gồm lỗi nhân (1) hoặc không lỗi (0). Bảng ?? tóm tắt chi tiết về các bộ dữ liệu được sử dụng.

Dữ liệu	Dự án	Số mẫu	Độ đo	Số mẫu NP	Số mẫu lỗi	Tỷ lệ mất cân bằng	Số mẫu cân bằng
PROMISE	CM1	498	21	449	49	9.84	628
	KC1	2109	21	1693	326	15.45	2856
	KC2	522	21	415	107	20.49	590
	PC1	1109	21	1032	77	6.9	1422
	JM1	10885	21	8779	2016	18.5	12288

Bảng 1: Kết quả mô hình dự đoán lỗi phần mềm không áp dụng kỹ thuật lấy mẫu và lựa chọn đặc trưng

Dataset	Performance Measures	RF	ET	AB	HGB
CM1	Precision	0.511	0.580	0.589	0.577
	Recall	0.502	0.556	0.561	0.542
	F1-score	0.502	0.563	0.567	0.546
	AUC	0.634	0.623	0.703	0.718
KC1	Precision	0.719	0.707	0.689	0.714
	Recall	0.579	0.623	0.609	0.640
	F1-score	0.596	0.645	0.630	0.663
	AUC	0.702	0.710	0.723	0.728
KC2	Precision	0.726	0.694	0.702	0.725
	Recall	0.665	0.646	0.671	0.678
	F1-score	0.684	0.660	0.683	0.695
	AUC	0.701	0.724	0.707	0.721
PC1	Precision	0.763	0.730	0.653	0.714
	Recall	0.594	0.593	0.616	0.620
	F1-score	0.629	0.624	0.629	0.649
	AUC	0.726	0.788	0.746	0.766
Average					
	Precision	0.679	0.677	0.658	0.682
	Recall	0.585	0.605	0.614	0.620
	F1-score	0.603	0.623	0.627	0.638
	AUC	0.690	0.711	0.719	0.733

5.4 Độ đo đánh giá

Để đánh giá mô hình dự đoán lỗi phần mềm, các độ đo đánh giá bao gồm false positive rate, độ chính xác (accuracy), precision, recall, balance and F-measure được áp dụng. Trước đó, các ký hiệu A, B, C, D được sử dụng trong đó:

Bảng 2: Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Chi-Squared and GAN.

Dataset	Performance				Chi-Squared								
	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.826	0.856	0.843	0.849	0.843	0.856	0.854	0.837	0.834	0.819	0.828	0.821
	Recall	0.712	0.824	0.780	0.810	0.860	0.878	0.848	0.846	0.804	0.832	0.726	0.816
	F1-score	0.758	0.834	0.806	0.827	0.849	0.861	0.849	0.840	0.816	0.824	0.766	0.817
	AUC	0.635	0.739	0.679	0.706	0.675	0.739	0.723	0.690	0.643	0.657	0.574	0.679
KC1	Precision	0.809	0.816	0.807	0.823	0.817	0.827	0.813	0.825	0.804	0.804	0.797	0.808
	Recall	0.759	0.826	0.802	0.831	0.845	0.848	0.841	0.850	0.814	0.808	0.815	0.802
	F1-score	0.778	0.820	0.804	0.826	0.820	0.830	0.817	0.830	0.803	0.802	0.801	0.799
	AUC	0.752	0.794	0.753	0.780	0.793	0.793	0.745	0.781	0.755	0.734	0.647	0.734
KC2	Precision	0.815	0.785	0.795	0.788	0.798	0.785	0.786	0.787	0.809	0.813	0.807	0.825
	Recall	0.809	0.798	0.798	0.796	0.815	0.806	0.8	0.806	0.638	0.672	0.657	0.668
	F1-score	0.811	0.788	0.794	0.788	0.792	0.788	0.788	0.791	0.665	0.698	0.682	0.693
	AUC	0.777	0.783	0.701	0.773	0.785	0.775	0.658	0.762	0.753	0.799	0.744	0.761
PC1	Precision	0.899	0.919	0.919	0.916	0.877	0.925	0.913	0.930	0.884	0.911	0.887	0.891
	Recall	0.824	0.916	0.907	0.916	0.922	0.935	0.921	0.938	0.817	0.814	0.682	0.823
	F1-score	0.854	0.917	0.911	0.915	0.897	0.925	0.916	0.929	0.837	0.836	0.739	0.844
	AUC	0.780	0.784	0.839	0.826	0.724	0.775	0.838	0.802	0.681	0.698	0.616	0.663
Average													
	Precision	0.837	0.844	0.841	0.844	0.833	0.848	0.841	0.845	0.833	0.837	0.830	0.836
	Recall	0.776	0.841	0.822	0.838	0.860	0.867	0.852	0.860	0.768	0.781	0.720	0.777
	F1-score	0.800	0.840	0.829	0.839	0.839	0.851	0.842	0.847	0.780	0.790	0.747	0.788
	AUC	0.736	0.775	0.743	0.771	0.744	0.771	0.741	0.759	0.708	0.722	0.645	0.709

1. A là số lượng mô-đun bị lỗi được dự đoán là lỗi.

2. B là số mô-đun bị lỗi được phân loại là không có lỗi.

3. C là số mô-đun không có lỗi được dự đoán là lỗi.

4. D là số mô-đun không có lỗi được dự đoán là không có lỗi.

1. Độ chính xác (accuracy): Tỷ lệ là số lượng mô-đun được dự đoán chính xác trên tổng số mô-đun.

$$accuracy = (A + D)/(A + B + C + D) \quad (8)$$

Từ phương trình trên có thể thấy độ chính xác bị ảnh hưởng bởi tỷ lệ cân bằng của lớp. Tuy nhiên, các bộ dữ liệu phần mềm thường thì nhiều mô-đun không có lỗi nhiều hơn các mô-đun bị lỗi. Nếu một mô hình dự đoán tất cả các mô-đun không bị lỗi, độ chính xác sẽ rất cao mặc dù không có mô-đun bị lỗi nào được dự đoán chính xác. Ví dụ, tỷ lệ lỗi trung bình của bộ dữ liệu PROMISE là 18%. Nếu một mô hình dự đoán tuyên bố tất cả các mô-đun là không có lỗi, độ chính xác của nó sẽ là 82% mà không dự đoán chính xác bất kỳ mô-đun bị lỗi nào. Do sự mất cân đối giữa các lớp trong tập dữ liệu, độ chính xác không thể được coi là thước đo thích hợp để so sánh các mô hình dự báo.

2. False positive rate: tỷ lệ báo động nhầm cũng được biết là xác suất báo động nhầm. Tỷ lệ này là số mô-đun không có lỗi được dự đoán sai như số lỗi của các mô-đun không có lỗi.

$$pf = C/(C + D) \quad (9)$$

3. Recall: Recall còn được gọi là dự báo đúng hoặc xác suất phát hiện (probability of detection). Recall được tính bằng tỷ lệ là số lượng mô-đun bị lỗi được dự đoán chính xác là bị lỗi với số lượng mô-đun bị lỗi.

$$recall = pd = A/(A + B) \quad (10)$$

4. Precision: Tỷ lệ là số lượng mô-đun bị lỗi được dự đoán chính xác là bị lỗi với số lượng mô-đun được dự đoán là lỗi.

$$precision = A/(A + C) \quad (11)$$

Một mô hình dự đoán trình bày một hiệu suất tốt nếu đạt được các giá trị thu hồi (recall), độ chính xác (accuracy) và giá trị thấp hơn của độ báo động giả (False positive rate). Tuy nhiên, người ta biết rằng việc recall có thể được cải thiện bằng cách giảm accuracy và ngược lại. Bởi vì sự cân bằng giữa accuracy và recall, không dễ so sánh các hiệu năng của bộ dự đoán lỗi dựa trên chỉ recall hoặc accuracy. Kết quả là, F- measure, một thước đo tổng hợp recall và accuracy, đã được sử dụng để so sánh kết quả dự đoán.

5. F- measure: Chức năng hài hòa của recall và accuracy

$$F - measure = (2 * recall * precision) / (recall + precision) \quad (12)$$

F-measure đã được sử dụng trong nhiều bài báo dự đoán lỗi. Độ đo này trình bày một điểm thống nhất để đánh giá mô hình dự đoán sau khi cân bằng sự cân bằng giữa recall và accuracy. Giá trị của F-measure tỷ lệ thuận với hiệu suất của một mô hình.

6. AUC: AUC biểu thị khu vực theo đường cong (ROC). AUC là một thước đo không tham số độc lập với ngưỡng và không bị ảnh hưởng bởi sự mất cân bằng trong lớp. ROC là một đường cong hai chiều được vẽ theo xác suất của báo động giả (trục x) và xác suất phát hiện (trục y). Theo Rahman và Devanbu một mô hình tốt hơn khi đường cong ROC của nó gần với điểm $pd = 1$ và $pf = 0$. Một bộ dự báo hoàn hảo, có AUC là 1. Ngược lại, một đường cong phủ định minh họa mô hình không tốt với xác suất báo động giả và xác suất phát hiện thấp.

5.5 Kết quả thực nghiệm

Trong nghiên cứu này, chúng tôi đã xây dựng các mô hình dự đoán lỗi bằng cách kết hợp các biến thể GAN khác nhau để tổng hợp các mẫu mới, sau đó sử dụng lựa chọn đặc trưng dựa trên filter/ wrapper để giảm các thuộc tính không liên quan hoặc dư thừa. Sau đó, chúng tôi sử dụng các mô hình học máy phổ biến và đánh giá tác động của sự mất cân bằng lớp và các thuộc tính dư thừa đối với hiệu suất của các mô hình này. Các thực nghiệm được thực hiện sử dụng 10-fold cross-validation cho mỗi thử nghiệm. Hiệu suất của các mô hình dự đoán lỗi đã phát triển được đánh giá bằng cách sử dụng các độ đo đánh giá về precision, recall, F1 và AUC.

Bảng 3: Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Information Gain và GAN.

Dataset	Performance	Information Gain											
	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.853	0.853	0.843	0.844	0.832	0.863	0.864	0.832	0.834	0.823	0.827	0.820
	Recall	0.722	0.840	0.790	0.812	0.870	0.888	0.866	0.850	0.858	0.852	0.782	0.790
	F1-score	0.769	0.843	0.812	0.826	0.847	0.865	0.861	0.839	0.844	0.836	0.801	0.802
	AUC	0.671	0.695	0.644	0.658	0.671	0.695	0.662	0.627	0.633	0.63	0.621	0.638
KC1	Precision	0.797	0.808	0.811	0.808	0.815	0.807	0.810	0.809	0.814	0.795	0.804	0.802
	Recall	0.836	0.830	0.841	0.832	0.845	0.832	0.840	0.836	0.827	0.796	0.807	0.789
	F1-score	0.801	0.815	0.818	0.816	0.822	0.815	0.814	0.817	0.817	0.792	0.804	0.793
	AUC	0.765	0.784	0.735	0.761	0.773	0.778	0.737	0.754	0.763	0.725	0.664	0.725
KC2	Precision	0.803	0.807	0.791	0.811	0.821	0.803	0.776	0.794	0.798	0.804	0.813	0.812
	Recall	0.804	0.815	0.791	0.813	0.825	0.815	0.787	0.808	0.64	0.643	0.628	0.634
	F1-score	0.800	0.807	0.789	0.807	0.807	0.801	0.778	0.796	0.661	0.668	0.655	0.657
	AUC	0.777	0.791	0.726	0.792	0.812	0.788	0.651	0.783	0.765	0.788	0.737	0.767
PC1	Precision	0.893	0.918	0.899	0.907	0.881	0.924	0.909	0.911	0.879	0.890	0.878	0.889
	Recall	0.892	0.915	0.907	0.917	0.919	0.932	0.922	0.922	0.799	0.877	0.818	0.858
	F1-score	0.891	0.915	0.902	0.911	0.897	0.925	0.913	0.914	0.829	0.876	0.838	0.866
	AUC	0.772	0.755	0.757	0.797	0.689	0.747	0.781	0.780	0.649	0.663	0.559	0.698
Average													
	Precision	0.836	0.846	0.836	0.843	0.837	0.849	0.840	0.836	0.831	0.828	0.830	0.830
	Recall	0.813	0.850	0.832	0.844	0.864	0.867	0.854	0.854	0.781	0.792	0.759	0.768
	F1-score	0.816	0.845	0.830	0.840	0.843	0.852	0.842	0.842	0.788	0.793	0.775	0.780
	AUC	0.746	0.753	0.716	0.752	0.736	0.753	0.708	0.736	0.702	0.702	0.645	0.707

5.5.1 Kết quả thực nghiệm 1

Như đã được trình bày trong phần lý thuyết, 3 phương pháp lấy mẫu oversampling khác nhau được sử dụng để cân bằng 4 bộ dữ liệu với độ đo phần mềm và 4 kỹ thuật lựa chọn đặc trưng dựa trên filter có được áp dụng để giải quyết vấn đề lựa chọn các thuộc tính tối ưu trong dự đoán lỗi. Hiệu suất của các mô hình dự đoán lỗi với kỹ thuật oversampling và lựa chọn đặc trưng dựa trên các dữ liệu đã chọn và đầy đủ độ đo phần mềm được thể hiện trong Bảng 2-6. Hiệu suất trung bình của bốn mô hình học máy (RF, ET, AB và HGB) cũng được tính toán để kiểm tra tính hiệu quả lựa chọn đặc trưng và lấy mẫu dữ liệu trong dự đoán lỗi về precision, recall, F1 và AUC. Từ các kết quả được trình bày trong Bảng 2-6, có thể thấy

Bảng 4: Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Fisher và GAN.

Performance		Fisher											
Dataset	Evaluation	VanillaGAN				CTGAN				WGANGP			
		RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.833	0.842	0.836	0.828	0.845	0.837	0.836	0.840	0.818	0.814	0.828	HgB
	Recall	0.674	0.838	0.766	0.794	0.850	0.866	0.816	0.852	0.788	0.852	0.746	0.832
	F1-score	0.734	0.839	0.794	0.810	0.844	0.848	0.824	0.845	0.799	0.832	0.778	0.820
	AUC	0.632	0.678	0.615	0.689	0.670	0.707	0.619	0.659	0.613	0.653	0.637	0.824
KC1	Precision	0.809	0.827	0.813	0.828	0.818	0.833	0.813	0.829	0.803	0.800	0.806	0.708
	Recall	0.785	0.839	0.834	0.844	0.843	0.855	0.839	0.851	0.800	0.792	0.783	0.805
	F1-score	0.794	0.832	0.819	0.833	0.823	0.836	0.819	0.835	0.798	0.795	0.793	0.773
	AUC	0.763	0.798	0.749	0.770	0.783	0.798	0.748	0.778	0.755	0.747	0.686	0.785
KC2	Precision	0.795	0.810	0.794	0.825	0.784	0.792	0.795	0.794	0.818	0.827	0.806	0.755
	Recall	0.783	0.811	0.794	0.830	0.8	0.811	0.809	0.815	0.675	0.664	0.628	0.829
	F1-score	0.787	0.808	0.793	0.826	0.778	0.793	0.799	0.799	0.700	0.690	0.649	0.674
	AUC	0.774	0.793	0.708	0.763	0.781	0.787	0.666	0.761	0.724	0.803	0.728	0.699
PC1	Precision	0.902	0.919	0.927	0.920	0.893	0.930	0.925	0.935	0.890	0.903	0.880	0.732
	Recall	0.900	0.93	0.932	0.926	0.914	0.939	0.932	0.942	0.846	0.877	0.732	0.896
	F1-score	0.900	0.922	0.926	0.922	0.899	0.932	0.927	0.935	0.865	0.883	0.785	0.851
	AUC	0.867	0.808	0.817	0.889	0.749	0.816	0.867	0.879	0.700	0.732	0.611	0.868
Average													
	Precision	0.835	0.848	0.842	0.850	0.835	0.848	0.842	0.849	0.832	0.836	0.830	0.840
	Recall	0.786	0.855	0.832	0.849	0.852	0.867	0.849	0.865	0.777	0.796	0.722	0.780
	F1-score	0.804	0.850	0.833	0.848	0.836	0.852	0.842	0.853	0.790	0.800	0.751	0.794
	AUC	0.759	0.769	0.722	0.778	0.746	0.776	0.725	0.769	0.698	0.734	0.666	0.733

rằng hiệu suất của phương pháp dự đoán các mô hình dựa trên sự kết hợp của 4 lựa chọn đặc trưng dựa trên filter và 2 mô hình lấy mẫu GAN (VanillaGAN và CTGAN) đã đạt kết quả tốt hơn so với khi xây dựng mô hình dự đoán lỗi không có lựa chọn đặc trưng và lấy mẫu dữ liệu được sử dụng (như thể hiện trong Bảng 1). Ví dụ: giá trị AUC trung bình của mô hình dự đoán lỗi trên các tập dữ liệu gốc là 0,690, 0,711, 0,719 và 0,733 với mô hình RF, ET, AB và HGB tương ứng.

Từ kết quả Bảng 5, có thể quan sát thấy Relief và CTGAN đạt được hiệu suất tốt hơn so với các kết hợp khác của kỹ thuật lấy mẫu và lựa chọn đặc trưng dựa trên filter khác với precision, recall, F1 và AUC trung bình cao nhất là 0,857 , 0,873, 0,856 và 0,767 tương ứng trên Extra Tree. Tuy nhiên, sự kết hợp của từng

Bảng 5: Kết quả mô hình dự đoán lỗi phần mềm dự trên kết hợp của Relief và GAN.

Performance		Relief											
Dataset		VanillaGAN				CTGAN				WGANGP			
	Evaluation	RF	ET	AB	HGB	RF	ET	AB	HGB	RF	ET	AB	HGB
CM1	Precision	0.845	0.848	0.843	0.835	0.824	0.872	0.855	0.844	0.829	0.834	0.839	0.831
	Recall	0.696	0.808	0.784	0.788	0.866	0.890	0.860	0.866	0.828	0.864	0.752	0.814
	F1-score	0.75	0.824	0.807	0.807	0.844	0.868	0.856	0.851	0.827	0.848	0.787	0.821
	AUC	0.647	0.716	0.672	0.673	0.676	0.711	0.716	0.670	0.648	0.637	0.661	0.690
KC1	Precision	0.815	0.824	0.811	0.827	0.815	0.833	0.825	0.843	0.815	0.821	0.816	0.824
	Recall	0.781	0.838	0.833	0.845	0.844	0.854	0.85	0.861	0.811	0.806	0.782	0.800
	F1-score	0.793	0.829	0.819	0.833	0.821	0.836	0.828	0.847	0.810	0.807	0.792	0.805
	AUC	0.773	0.798	0.737	0.776	0.788	0.797	0.757	0.788	0.759	0.746	0.720	0.740
KC2	Precision	0.811	0.789	0.789	0.798	0.781	0.802	0.785	0.789	0.816	0.809	0.797	0.812
	Recall	0.794	0.804	0.791	0.804	0.804	0.817	0.798	0.806	0.658	0.651	0.609	0.642
	F1-score	0.800	0.793	0.788	0.799	0.78	0.800	0.789	0.792	0.682	0.675	0.634	0.665
	AUC	0.796	0.789	0.705	0.772	0.781	0.783	0.652	0.772	0.780	0.791	0.738	0.760
PC1	Precision	0.900	0.913	0.916	0.915	0.871	0.922	0.914	0.929	0.893	0.902	0.893	0.898
	Recall	0.859	0.917	0.905	0.913	0.914	0.932	0.921	0.937	0.794	0.770	0.695	0.759
	F1-score	0.873	0.914	0.909	0.912	0.89	0.921	0.915	0.928	0.823	0.798	0.747	0.800
	AUC	0.782	0.783	0.839	0.814	0.691	0.775	0.841	0.802	0.680	0.707	0.646	0.685
Average													
	Precision	0.843	0.843	0.840	0.844	0.822	0.857	0.845	0.851	0.838	0.841	0.836	0.841
	Recall	0.783	0.842	0.828	0.837	0.857	0.873	0.857	0.867	0.773	0.773	0.710	0.753
	F1-score	0.804	0.840	0.831	0.838	0.834	0.856	0.847	0.855	0.785	0.782	0.740	0.773
	AUC	0.757	0.767	0.738	0.759	0.734	0.767	0.742	0.758	0.717	0.720	0.691	0.719

kỹ thuật lựa chọn đặc trưng (Chi-Squared, In- đội hình Gain, Fisher, Relief) và WGAN tạo ra hiệu suất tương tự khi không có kỹ thuật lựa chọn đặc trưng và lấy mẫu dữ liệu nào được áp dụng. Khi so sánh các hiệu suất của phương pháp WGANP với kỹ thuật AdaBoost, chúng tôi thấy rằng giá trị AUC trung bình của Chi-Squared, Information Gain, Fisher và Relief có giá trị AUC thấp nhất với lần lượt là 0,645, 0,645, 0,666 và 0,691 như trong Bảng 2-6.

5.5.2 Kết quả thực nghiệm 2

Chúng tôi sử dụng tập dữ liệu huấn luyện và kiểm tra riêng biệt cho mỗi lần lặp lại thực nghiệm 10 lần; kết quả cuối cùng là trung bình của 10 lần lặp. Kết quả

Bảng 6: Kết quả so sánh của kết hợp các kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và kỹ luật lấy mẫu VanillaGAN.

Dataset	Performance	GA+VanillaGAN					PSO+VanillaGAN					WOA + VanillaGAN					Without FS				
	Evaluation	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR
CM1	Precision	0.808	0.790	0.804	0.750	0.859	0.806	0.813	0.81	0.805	0.833	0.821	0.782	0.808	0.803	0.829	0.792	0.780	0.801	0.749	0.800
	Recall	0.726	0.829	0.811	0.734	0.786	0.740	0.860	0.849	0.863	0.686	0.754	0.860	0.837	0.866	0.706	0.707	0.825	0.811	0.803	0.876
	F1-score	0.758	0.808	0.806	0.741	0.820	0.768	0.832	0.827	0.831	0.738	0.783	0.819	0.819	0.833	0.752	0.747	0.801	0.805	0.775	0.826
	AUC	0.617	0.688	0.646	0.509	0.690	0.689	0.740	0.645	0.524	0.687	0.675	0.729	0.617	0.543	0.720	0.608	0.683	0.637	0.506	0.686
KC1	Precision	0.813	0.814	0.793	0.831	0.824	0.822	0.829	0.818	0.837	0.821	0.825	0.832	0.818	0.834	0.824	0.810	0.819	0.816	0.815	0.801
	Recall	0.809	0.833	0.828	0.778	0.670	0.831	0.855	0.847	0.657	0.666	0.836	0.854	0.852	0.706	0.697	0.802	0.830	0.819	0.697	0.656
	F1-score	0.811	0.818	0.804	0.797	0.713	0.825	0.817	0.819	0.701	0.710	0.828	0.816	0.821	0.741	0.735	0.805	0.810	0.813	0.751	0.701
	AUC	0.726	0.789	0.769	0.783	0.755	0.746	0.817	0.795	0.769	0.744	0.741	0.804	0.790	0.780	0.766	0.715	0.785	0.763	0.764	0.799
KC2	Precision	0.869	0.876	0.882	0.879	0.884	0.881	0.879	0.870	0.871	0.876	0.874	0.879	0.876	0.884	0.872	0.866	0.863	0.866	0.872	0.870
	Recall	0.841	0.875	0.862	0.745	0.693	0.866	0.924	0.913	0.900	0.789	0.866	0.922	0.913	0.907	0.793	0.817	0.884	0.857	0.879	0.769
	F1-score	0.853	0.875	0.870	0.793	0.761	0.872	0.894	0.889	0.884	0.826	0.869	0.894	0.892	0.893	0.827	0.840	0.870	0.861	0.875	0.816
	AUC	0.719	0.76	0.721	0.707	0.707	0.739	0.785	0.715	0.672	0.658	0.729	0.783	0.727	0.671	0.635	0.710	0.759	0.701	0.67	0.637
PC1	Precision	0.817	0.818	0.788	0.819	0.828	0.797	0.811	0.801	0.820	0.824	0.814	0.808	0.798	0.806	0.832	0.812	0.807	0.782	0.81	0.819
	Recall	0.798	0.813	0.792	0.828	0.755	0.783	0.825	0.819	0.823	0.749	0.798	0.825	0.817	0.817	0.796	0.727	0.810	0.803	0.82	0.722
	F1-score	0.804	0.811	0.784	0.816	0.774	0.789	0.811	0.789	0.818	0.769	0.800	0.806	0.797	0.808	0.807	0.767	0.808	0.792	0.814	0.767
	AUC	0.772	0.804	0.738	0.817	0.820	0.760	0.826	0.817	0.818	0.817	0.788	0.806	0.798	0.772	0.819	0.770	0.767	0.737	0.74	0.809
JM1	Precision	0.747	0.766	0.758	0.768	0.759	0.747	0.762	0.757	0.764	0.750	0.742	0.749	0.747	0.762	0.761	0.738	0.747	0.742	0.756	0.740
	Recall	0.768	0.809	0.806	0.796	0.704	0.774	0.807	0.805	0.785	0.555	0.768	0.770	0.776	0.796	0.591	0.766	0.760	0.777	0.785	0.511
	F1-score	0.756	0.760	0.755	0.777	0.725	0.758	0.756	0.754	0.771	0.595	0.753	0.753	0.751	0.771	0.631	0.751	0.759	0.753	0.770	0.541
	AUC	0.658	0.708	0.702	0.663	0.668	0.659	0.708	0.700	0.674	0.643	0.651	0.69	0.684	0.68	0.674	0.674	0.679	0.681	0.642	0.638
Average results																					
	Precision	0.811	0.813	0.805	0.809	0.831	0.811	0.819	0.811	0.819	0.821	0.815	0.810	0.809	0.818	0.824	0.804	0.803	0.801	0.800	0.806
	Recall	0.788	0.832	0.820	0.776	0.722	0.799	0.854	0.847	0.806	0.689	0.804	0.846	0.839	0.818	0.717	0.764	0.822	0.813	0.797	0.707
	F1-score	0.796	0.814	0.804	0.785	0.759	0.802	0.822	0.816	0.801	0.728	0.807	0.818	0.816	0.809	0.750	0.782	0.810	0.805	0.797	0.730
	AUC	0.698	0.750	0.715	0.696	0.728	0.719	0.775	0.734	0.691	0.710	0.717	0.762	0.723	0.689	0.723	0.695	0.735	0.704	0.664	0.714

FS: Feature Selection

đã được báo cáo trong Bảng 6 và Bảng 7, bao gồm năm kỹ thuật học máy cho kết quả trung bình trong số năm bộ dữ liệu của mỗi phương pháp Wrapper tương ứng các độ đo hiệu suất được sử dụng. Đối với độ đo đánh giá Precision, các giá trị đạt được hầu như cao hơn 0.80 cho tất cả các kỹ thuật ngoại trừ tập dữ liệu JM1, giá trị cao nhất là 0.884 đối với tập dữ liệu PC1 trong cặp GA và WOA và giá trị thấp nhất là 0.732 đối với tập dữ liệu JM1 trong kỹ thuật MA. Đối với độ đo Recall, giá trị gần như cao hơn 0.80, cặp PSO và CS đạt mức cao nhất giá trị 0.924 đối với PC1 và giá trị thấp nhất là 0.591 đối với tập dữ liệu JM1 trong kỹ thuật WOA. Đối với độ đo F1, BBA đạt giá trị cao nhất là 0.895 cho tập dữ liệu PC1, PSO và CS đạt giá trị 0.894, cũng là mức tốt và giá trị thấp nhất là 0.40 đối với tập dữ liệu CM1. Về mặt đo lường AUC, trong hầu hết các trường hợp, giá trị lớn hơn 0.700 ở CM1, PC1, KC1 và KC2. Giá trị AUC cao nhất là 0,826

Bảng 7: Kết quả so sánh của kết hợp các kỹ thuật lựa chọn đặc trưng dựa trên Wrapper và kỹ luật lấy mẫu VanillaGAN (tt).

Dataset	Performance	MA+VanillaGAN					CS+VanillaGAN					BBA+ VanillaGAN					Without FS				
	Evaluation	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR
CM1	Precision	0.814	0.809	0.804	0.787	0.844	0.819	0.818	0.817	0.785	0.847	0.82	0.794	0.798	0.789	0.836	0.792	0.780	0.801	0.749	0.800
	Recall	0.737	0.866	0.843	0.854	0.726	0.731	0.874	0.846	0.854	0.723	0.771	0.854	0.834	0.851	0.72	0.707	0.825	0.811	0.803	0.876
	F1-score	0.769	0.83	0.82	0.818	0.768	0.766	0.839	0.828	0.816	0.766	0.792	0.82	0.813	0.817	0.763	0.747	0.801	0.805	0.775	0.826
	AUC	0.676	0.708	0.644	0.542	0.727	0.683	0.744	0.671	0.568	0.715	0.686	0.727	0.623	0.538	0.689	0.608	0.683	0.637	0.506	0.686
KC1	Precision	0.818	0.84	0.825	0.828	0.824	0.829	0.823	0.815	0.833	0.819	0.817	0.834	0.817	0.813	0.824	0.810	0.819	0.816	0.815	0.801
	Recall	0.829	0.859	0.853	0.721	0.688	0.829	0.853	0.848	0.666	0.673	0.827	0.858	0.849	0.663	0.673	0.802	0.830	0.819	0.697	0.656
	F1-score	0.822	0.827	0.82	0.752	0.728	0.818	0.814	0.813	0.709	0.715	0.821	0.827	0.818	0.694	0.716	0.805	0.810	0.813	0.751	0.701
	AUC	0.742	0.807	0.782	0.782	0.771	0.742	0.805	0.779	0.776	0.747	0.743	0.812	0.784	0.767	0.757	0.715	0.785	0.763	0.764	0.799
KC2	Precision	0.876	0.879	0.871	0.879	0.878	0.87	0.879	0.87	0.878	0.874	0.882	0.877	0.87	0.879	0.879	0.866	0.863	0.866	0.872	0.870
	Recall	0.859	0.922	0.914	0.904	0.789	0.854	0.924	0.913	0.897	0.779	0.868	0.924	0.909	0.897	0.786	0.817	0.884	0.857	0.879	0.769
	F1-score	0.867	0.894	0.89	0.89	0.827	0.861	0.894	0.889	0.887	0.818	0.874	0.895	0.887	0.886	0.824	0.840	0.870	0.861	0.875	0.816
	AUC	0.717	0.789	0.716	0.665	0.659	0.716	0.79	0.72	0.675	0.652	0.736	0.801	0.719	0.669	0.652	0.710	0.759	0.701	0.67	0.637
PC1	Precision	0.797	0.807	0.782	0.817	0.823	0.783	0.824	0.829	0.826	0.83	0.827	0.815	0.785	0.812	0.828	0.812	0.807	0.782	0.81	0.819
	Recall	0.774	0.823	0.794	0.825	0.747	0.758	0.819	0.825	0.832	0.766	0.809	0.826	0.806	0.821	0.787	0.727	0.810	0.803	0.82	0.722
	F1-score	0.782	0.811	0.785	0.815	0.768	0.767	0.818	0.824	0.822	0.784	0.815	0.816	0.792	0.81	0.798	0.767	0.808	0.792	0.814	0.767
	AUC	0.75	0.787	0.735	0.815	0.815	0.747	0.805	0.783	0.826	0.819	0.806	0.808	0.754	0.781	0.812	0.770	0.767	0.737	0.74	0.809
JM1	Precision	0.747	0.742	0.732	0.756	0.757	0.76	0.76	0.757	0.767	0.761	0.76	0.764	0.764	0.767	0.745	0.738	0.747	0.742	0.756	0.740
	Recall	0.778	0.757	0.762	0.777	0.593	0.787	0.806	0.803	0.793	0.725	0.784	0.8	0.801	0.803	0.604	0.766	0.760	0.777	0.785	0.511
	F1-score	0.756	0.745	0.741	0.764	0.63	0.769	0.758	0.759	0.776	0.74	0.769	0.765	0.76	0.774	0.642	0.751	0.759	0.753	0.770	0.541
	AUC	0.654	0.679	0.668	0.655	0.669	0.666	0.72	0.709	0.663	0.672	0.686	0.709	0.703	0.682	0.651	0.674	0.679	0.681	0.642	0.638
Average results																					
	Precision	0.81	0.815	0.803	0.813	0.825	0.812	0.821	0.818	0.818	0.826	0.821	0.817	0.807	0.812	0.822	0.804	0.803	0.801	0.800	0.806
	Recall	0.795	0.845	0.833	0.816	0.709	0.792	0.855	0.847	0.808	0.733	0.812	0.852	0.84	0.807	0.714	0.764	0.822	0.813	0.797	0.707
	F1-score	0.799	0.821	0.811	0.808	0.744	0.796	0.825	0.823	0.802	0.765	0.814	0.825	0.814	0.796	0.749	0.782	0.810	0.805	0.797	0.730
	AUC	0.708	0.754	0.709	0.692	0.728	0.711	0.773	0.732	0.702	0.721	0.731	0.771	0.717	0.687	0.712	0.695	0.735	0.704	0.664	0.714

FS: Feature Selection

đối với KC2 trong phương pháp PSO và giá trị thấp nhất là 0,509 đối với CM1. Ngoài ra, hiệu suất của các bộ phân loại dựa trên các phương pháp lựa chọn tính năng Wrapper tốt hơn so với với tập dữ liệu gốc. Từ kết quả trung bình, có thể thấy rằng PSO với Vanilla GAN và Cuckoo Search với VanillaGAN tạo ra các giá trị tốt nhất và cao hơn các trường hợp khác trong hầu hết các trường hợp, tiếp theo là Thuật toán Mayfly với Vanilla GAN.

6 Kết luận

Dữ liệu nhiều chiều và các lớp không cân bằng là những yếu tố chính ảnh hưởng đáng kể đến hiệu suất của các mô hình dự đoán lỗi phần mềm. Các kỹ thuật lựa chọn đặc trưng, chọn các tập hợp con dữ liệu tối ưu, được áp dụng để giảm các

thuộc tính không liên quan đến lỗi và dư thừa. Các phương pháp lấy mẫu dữ liệu nhằm tạo ra các thể hiện của lớp mẫu mới và dữ liệu cân bằng đã được nhiều nhà nghiên cứu đề xuất để giải quyết vấn đề này. Trong nghiên cứu này, chúng tôi thực hiện phát triển các mô hình dự đoán lỗi hiệu quả bằng cách xử lý giải quyết vấn đề mất cân bằng lớp và dư thừa đặc trưng một cách hiệu quả. Các biến thể GAN đại diện cho các kỹ thuật tiên tiến (CTGAN, Vanilla GAN và WANGGP) được áp dụng để lấy mẫu dữ liệu khi xử lý dữ liệu mất cân bằng. Sau đó, chúng tôi sử dụng các kỹ thuật lựa chọn đặc trưng (lựa chọn đặc trưng dựa trên filter và wrapper) trên các tập dữ liệu cân bằng để lựa chọn các đặc trưng tối ưu nhất trong dự đoán lỗi. Các mô hình dự đoán lỗi được phát triển bằng cách sử dụng các mô hình phân loại khác nhau sẽ được phân tích và hiệu suất của mô hình này được thực hiện trên 5 bộ dữ liệu nguồn mở. Kết quả thử nghiệm cho thấy hiệu suất của các mô hình dự đoán lỗi dựa trên sự kết hợp của Relief&CTGAN, PSO&VanillaGAN và Cuckoo Search&VanillaGAN đạt được hiệu suất cao nhất trên hầu hết các tập dữ liệu. Trong tương lai, chúng tôi dự định khái quát hóa những phát hiện của các mô hình đã trình bày để tích hợp nhiều phương pháp lựa chọn đặc trưng và kỹ thuật lấy mẫu dữ liệu bằng cách sử dụng nhiều bộ dữ liệu lỗi phần mềm hơn.

References

- [1] Aleksei Guryanov. “Histogram-based algorithm for building gradient boosting ensembles of piecewise linear decision trees”. In: *Analysis of Images, Social Networks and Texts: 8th International Conference, AIST 2019, Kazan, Russia, July 17–19, 2019, Revised Selected Papers 8*. Springer. 2019, pp. 39–50.
- [2] Antonia Creswell et al. “Generative adversarial networks: An overview”. In: *IEEE signal processing magazine* 35.1 (2018), pp. 53–65.
- [3] Avrim L Blum and Pat Langley. “Selection of relevant features and examples in machine learning”. In: *Artificial intelligence* 97.1-2 (1997), pp. 245–271.
- [4] Bartosz Krawczyk. “Learning from imbalanced data: open challenges and future directions”. In: *Progress in Artificial Intelligence* 5.4 (2016), pp. 221–232.
- [5] Cagatay Catal and Banu Diri. “A systematic review of software fault prediction studies”. In: *Expert Systems with Applications* 36.4 (2009), pp. 7346–7354. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2008.10.027>. URL: <https://www.sciencedirect.com/science/article/pii/S0957417408007215>.
- [6] Chakkrit Tantithamthavorn, Ahmed E Hassan, and Kenichi Matsumoto. “The impact of class rebalancing techniques on the performance and interpretation of defect prediction models”. In: *IEEE Transactions on Software Engineering* 46.11 (2018), pp. 1200–1219.
- [7] Chris Seiffert et al. “RUSBoost: A hybrid approach to alleviating class imbalance”. In: *IEEE transactions on systems, man, and cybernetics-part A: systems and humans* 40.1 (2009), pp. 185–197.
- [8] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006. ISBN: 0387310738.

- [9] Fei Wang, Jun Ai, and Zhuoliang Zou. “A cluster-based hybrid feature selection method for defect prediction”. In: *2019 IEEE 19th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE. 2019, pp. 1–9.
- [10] Girish Chandrashekar and Ferat Sahin. “A survey on feature selection methods”. In: *Computers & Electrical Engineering* 40.1 (2014), pp. 16–28.
- [11] Haidar Osman, Mohammad Ghafari, and Oscar Nierstrasz. “Automatic feature selection by regularization to improve bug prediction accuracy”. In: *2017 IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTaSQuE)*. IEEE. 2017, pp. 27–32.
- [12] Hamoud Aljamaan and Amal Alazba. “Software Defect Prediction Using Tree-Based Ensembles”. In: *Proceedings of the 16th ACM International Conference on Predictive Models and Data Analytics in Software Engineering*. PROMISE 2020. Virtual, USA: Association for Computing Machinery, 2020, pp. 1–10. ISBN: 9781450381277. DOI: 10 . 1145 / 3416508 . 3417114. URL: <https://doi.org/10.1145/3416508.3417114>.
- [13] Haonan Tong, Bin Liu, and Shihai Wang. “Software defect prediction using stacked denoising autoencoders and two-stage ensemble learning”. In: *Information and Software Technology* 96 (2018), pp. 94–111.
- [14] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. “Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning”. In: *International conference on intelligent computing*. Springer. 2005, pp. 878–887.
- [15] Ian Goodfellow et al. “Advances in neural information processing systems”. In: *Curran Associates, Inc* 27 (2014), pp. 2672–2680.
- [16] Isabelle Guyon and André Elisseeff. “An introduction to variable and feature selection”. In: *Journal of machine learning research* 3.Mar (2003), pp. 1157–1182.
- [17] Ishaan Gulrajani et al. “Improved Training of Wasserstein GANs”. In: *Proceedings of the 31st International Conference on Neural Information Process-*

- ing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 5769–5779. ISBN: 9781510860964.
- [18] Iyad Tumar et al. “Enhanced binary moth flame optimization as a feature selection algorithm to predict software fault prediction”. In: *Ieee Access* 8 (2020), pp. 8041–8055.
 - [19] Jaechang Nam and Sunghun Kim. “CLAMI: Defect Prediction on Unlabeled Datasets (T)”. In: *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2015, pp. 452–463. DOI: 10.1109/ASE.2015.56.
 - [20] Jaechang Nam et al. “Heterogeneous Defect Prediction”. In: *IEEE Transactions on Software Engineering* 44.9 (2018), pp. 874–896. DOI: 10.1109/TSE.2017.2720603.
 - [21] Jelber Sayyad Shirabad and Tim Menzies. “The PROMISE Repository of Software Engineering Databases.” In: 2005.
 - [22] Kai Ming Ting. “An instance-weighting method to induce cost-sensitive trees”. In: *IEEE Transactions on Knowledge and Data Engineering* 14.3 (2002), pp. 659–665.
 - [23] Kwabena Ebo Bennin et al. “Mahakil: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction”. In: *IEEE Transactions on Software Engineering* 44.6 (2017), pp. 534–550.
 - [24] Kwabena Ebo Bennin et al. “The significant effects of data sampling approaches on software defect prioritization and classification”. In: *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE. 2017, pp. 364–373.
 - [25] Lei Qiao et al. “Deep learning based software defect prediction”. In: *Neurocomputing* 385 (2020), pp. 100–110.
 - [26] Lei Xu and Kalyan Veeramachaneni. “Synthesizing tabular data using generative adversarial networks”. In: *arXiv preprint arXiv:1811.11264* (2018).

- [27] Lei Xu et al. “Modeling Tabular Data Using Conditional GAN”. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., 2019.
- [28] Lillian J Ratliff, Samuel A Burden, and S Shankar Sastry. “Characterization and computation of local Nash equilibria in continuous games”. In: *2013 51st Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE. 2013, pp. 917–924.
- [29] Lina Gong, Shujuan Jiang, and Li Jiang. “Tackling class imbalance problem in software defect prediction through cluster-based over-sampling with filtering”. In: *IEEE Access* 7 (2019), pp. 145725–145737.
- [30] Mardhiya Hayaty, Siti Muthmainah, and Syed Muhammad Ghufuran. “Random and synthetic over-sampling approach to resolve data imbalance in classification”. In: *International Journal of Artificial Intelligence Research* 4.2 (2020), pp. 86–94.
- [31] Mark Lorenz and Jeff Kidd. *Object-Oriented Software Metrics: A Practical Guide*. USA: Prentice-Hall, Inc., 1994. ISBN: 013179292X.
- [32] Martin Shepperd et al. “Data quality: Some comments on the nasa software defect datasets”. In: *IEEE Transactions on software engineering* 39.9 (2013), pp. 1208–1215.
- [33] Maurice H. Halstead. *Elements of Software Science (Operating and Programming Systems Series)*. USA: Elsevier Science Inc., 1977. ISBN: 0444002057.
- [34] Meiliana et al. “Software metrics for fault prediction using machine learning approaches: A literature review with PROMISE repository dataset”. In: *2017 IEEE International Conference on Cybernetics and Computational Intelligence (CyberneticsCom)*. 2017, pp. 19–23. DOI: 10.1109/CYBERNETICSCOM.2017.8311708.
- [35] Mengmeng Zhu and Hoang Pham. “A two-phase software reliability modeling involving with software fault dependency and imperfect fault removal”. In: *Computer Languages, Systems & Structures* 53 (2018), pp. 27–42.

- [36] Michael J Siers and Md Zahidul Islam. “Novel algorithms for cost-sensitive classification and knowledge discovery in class imbalanced datasets with an application to NASA software defects”. In: *Information Sciences* 459 (2018), pp. 53–70.
- [37] Mikel Galar et al. “A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42.4 (2011), pp. 463–484.
- [38] Ming Tan et al. “Online defect prediction for imbalanced data”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 2. IEEE. 2015, pp. 99–108.
- [39] Nathalie Japkowicz and Shaju Stephen. “The class imbalance problem: A systematic study”. In: *Intelligent data analysis* 6.5 (2002), pp. 429–449.
- [40] NC Shrikanth and Tim Menzies. “Assessing practitioner beliefs about software defect prediction”. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*. 2020, pp. 182–190.
- [41] Nitesh V Chawla et al. “SMOTE: synthetic minority over-sampling technique”. In: *Journal of artificial intelligence research* 16 (2002), pp. 321–357.
- [42] Noelia Sánchez-Marroño, Amparo Alonso-Betanzos, and María Tombilla-Sanromán. “Filter methods for feature selection—a comparative study”. In: *Lecture notes in computer science* 4881 (2007), pp. 178–187.
- [43] Pierre Geurts, Damien Ernst, and Louis Wehenkel. “Extremely randomized trees”. In: *Machine learning* 63 (2006), pp. 3–42.
- [44] Qinbao Song, Yuchen Guo, and Martin Shepperd. “A comprehensive investigation of the role of imbalanced learning for software defect prediction”. In: *IEEE Transactions on Software Engineering* 45.12 (2018), pp. 1253–1269.

- [45] R Muthukrishnan and R Rohini. “LASSO: A feature selection technique in predictive modeling for machine learning”. In: *2016 IEEE international conference on advances in computer applications (ICACA)*. IEEE. 2016, pp. 18–20.
- [46] Rohit Singh and Santosh Singh Rathore. “Linear and non-linear bayesian regression methods for software fault prediction”. In: *International Journal of System Assurance Engineering and Management* 13.4 (2022), pp. 1864–1884.
- [47] Ron Kohavi and George H John. “Wrappers for feature subset selection”. In: *Artificial intelligence* 97.1-2 (1997), pp. 273–324.
- [48] Ruchika Malhotra and Shine Kamal. “An empirical study to investigate over-sampling methods for improving software defect prediction using imbalanced data”. In: *Neurocomputing* 343 (2019), pp. 120–140.
- [49] Ruchika Malhotra. “A systematic review of machine learning techniques for software fault prediction”. In: *Applied Soft Computing* 27 (2015), pp. 504–518.
- [50] Ruud JG Van Sloun et al. “Deep learning for super-resolution vascular ultrasound imaging”. In: *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2019, pp. 1055–1059.
- [51] S.R. Chidamber and C.F. Kemerer. “A metrics suite for object oriented design”. In: *IEEE Transactions on Software Engineering* 20.6 (1994), pp. 476–493. DOI: 10.1109/32.295895.
- [52] Safa Omri and Carsten Sinz. “Deep Learning for Software Defect Prediction: A Survey”. In: *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*. ICSEW’20. Seoul, Republic of Korea: Association for Computing Machinery, 2020, pp. 209–214. ISBN: 9781450379632. DOI: 10.1145/3387940.3391463. URL: <https://doi.org/10.1145/3387940.3391463>.

- [53] Santosh S Rathore and Sandeep Kumar. “A study on software fault prediction techniques”. In: *Artificial Intelligence Review* 51 (2019), pp. 255–327.
- [54] Santosh Singh Rathore et al. “Generative Oversampling Methods for Handling Imbalanced Data in Software Fault Prediction”. In: *IEEE Transactions on Reliability* 71.2 (2022), pp. 747–762.
- [55] Shamsul Huda et al. “An ensemble oversampling model for class imbalance problem in software defect prediction”. In: *IEEE access* 6 (2018), pp. 24184–24195.
- [56] Shuo Feng et al. “COSTE: Complexity-based OverSampling TEchnique to alleviate the class imbalance problem in software defect prediction”. In: *Information and Software Technology* 129 (2021), p. 106432.
- [57] Simon Haykin. *Neural networks: a comprehensive foundation*. Prentice Hall PTR, 1998.
- [58] Sina Fathi-Kazerooni and Roberto Rojas-Cessa. “Gan tunnel: Network traffic steganography by using gans to counter internet traffic classifiers”. In: *Ieee Access* 8 (2020), pp. 125345–125359.
- [59] T.J. McCabe. “A Complexity Measure”. In: *IEEE Transactions on Software Engineering* SE-2.4 (1976), pp. 308–320. DOI: 10.1109/TSE.1976.233837.
- [60] Tero Karras et al. “Progressive growing of gans for improved quality, stability, and variation”. In: *arXiv preprint arXiv:1710.10196* (2017).
- [61] Thomas Zimmermann, Rahul Premraj, and Andreas Zeller. “Predicting defects for eclipse”. In: *Third International Workshop on Predictor Models in Software Engineering (PROMISE’07: ICSE Workshops 2007)*. IEEE. 2007, pp. 9–9.
- [62] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’16. San Francisco, California, USA: Association for Computing Machinery, 2016, pp. 785–794. ISBN:

9781450342322. DOI: 10.1145/2939672.2939785. URL: <https://doi.org/10.1145/2939672.2939785>.

- [63] Tracy Hall and David Bowes. “The state of machine learning methodology in software fault prediction”. In: *2012 11th international conference on machine learning and applications*. Vol. 2. IEEE. 2012, pp. 308–313.
- [64] Tracy Hall et al. “A Systematic Literature Review on Fault Prediction Performance in Software Engineering”. In: *IEEE Transactions on Software Engineering* 38 (2012), pp. 1276–1304.
- [65] Yasutaka Kamei et al. “The effects of over and under sampling on fault-prone module detection”. In: *First international symposium on empirical software engineering and measurement (ESEM 2007)*. IEEE. 2007, pp. 196–204.
- [66] Yibiao Yang et al. “Effort-Aware Just-in-Time Defect Prediction: Simple Unsupervised Models Could Be Better than Supervised Models”. In: *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. FSE 2016. Seattle, WA, USA: Association for Computing Machinery, 2016, pp. 157–168. ISBN: 9781450342186. DOI: 10.1145/2950290.2950353. URL: <https://doi.org/10.1145/2950290.2950353>.
- [67] Yoav Freund. “Boosting a weak learning algorithm by majority”. In: *Information and computation* 121.2 (1995), pp. 256–285.
- [68] Zhanqi Cui et al. “Improving software fault localization by combining spectrum and mutation”. In: *IEEE Access* 8 (2020), pp. 172296–172307.
- [69] Zhiqiang Li, Xiao-Yuan Jing, and Xiaoke Zhu. “Progress on approaches to software defect prediction”. In: *Iet Software* 12.3 (2018), pp. 161–175.
- [70] Zhiyuan Wan et al. “Perceptions, expectations, and challenges in defect prediction”. In: *IEEE Transactions on Software Engineering* 46.11 (2018), pp. 1241–1266.
- [71] URL: <http://promise.site.uottawa.ca/SERepository/datasets-page.html>.

A comparative study of Wrapper feature selection techniques in Software Fault Prediction

Nguyen Thanh Long¹ and Ha Thi Minh Phuong² Nguyen Thanh Binh³[0000-0002-0154-1162]

¹ The University of Danang - University of Science and Technology

² ³ The University of Danang - Vietnam-Korea University of Information and Communication Technology, Danang

ngthlo.doc@gmail.com

htmpuong@vku.udn.vn

ntbinh@vku.udn.vn

Abstract. Software fault prediction aims to classify whether the module is defective or not-defective. In software systems, there are some software metrics may contain irrelevant or redundant information that leads to negative impact on the performance of the fault prediction model. Therefore, feature selection is an method that several studies have addressed to reduce computation time, improve prediction performance and provide a better understanding of data in machine learning. Additionally, the presence of imbalanced classes is one of the most challenge in software fault prediction. In this study, we examined the effectiveness of six different wrapper feature selection including Genetic Algorithm, Particle Swarm Optimization, Whale Optimization Algorithm, Cuckoo Search, Mayfly Algorithm and Binary Bat Algorithm for selecting the optimal subset of features. Then, we applied VanilaGAN to train the dataset with optimal features for handling the imbalanced problem. Subsequently, these generated training dataset and the tesing dataset are fed to the machine learning techniques. Experimental validation has been done on five dataset collected from Promise repository and Precision, Recall, F1-score, and AUC are evaluation performance measurements.

Keywords: Software fault prediction, feature selection, Wrapper, VanilaGAN, dataset

1 Introduction

Software defect prediction (SDP) is a significant procedure in software engineering that aims to identify potential defects in software systems before they occur. The robust software predictive model predicts correctly bugs or defects before releasing a new software version. Therefore, developers can effectively plan the distribution of testing effort to reduce the overall cost of software development and improve the quality and reliability of the final product.

In recent studies, machine learning techniques have been exploited widely

to construct a fault prediction model. These techniques use the historical defect dataset that has been collected from previous software projects to make predictions about the occurrence of future defects. The defect dataset consists of various software features (software metrics) e.g., Response for a Class, Depth of inheritance tree or Line of Code and labels which indicates defective or not for each module. During training, the predictive model learns the characteristics of software projects and makes predictions whether new module is faulty or not faulty [24]. Several researches state that the performance of software fault prediction models depends on software fault dataset [14, 2]. However, there are two major challenges that the software defect prediction dataset faces the high dimensional features and imbalanced classes [22]. Additionally, some software metrics are irrelevant or redundant to fault-proneness modules. They are the most significant reasons that reduce the effectiveness of used machine learning techniques in SFP.

In SFP literature, feature selection is employed to address the problem of minimizing the redundant and irrelevant features that are not useful for prediction. The objective of feature selection is to generate a subset of optimal metrics from the input data to achieve better prediction performance [5]. Numerous feature selection techniques have been proposed in software fault prediction domain [10, 19]. Feature selection techniques can be broadly categorized into three types, namely filter, wrapper and embedded techniques. The filter technique involves applying a statistical measure to each feature and selecting the top features based on their scores. Wrapper methods use a machine learning algorithm to evaluate the performance of a subset of features. The algorithm is trained using different subsets of features and the subset that yields the best performance is selected. Embedded methods incorporate feature selection into the learning algorithm itself. For example, some machine learning algorithms have built-in feature selection mechanisms, such as Lasso regression, decision trees, and random forests. According to Savina Colaco et al. [6], wrapper-based technique is the most popular use across all feature selection methods.

In this study, we examine the performance of different wrapper feature selection methods including Genetic Algorithm (GA), The Particle Swarm Optimization (PSO), Whale Optimization Algorithm (WOA), Cuckoo Search (CS), Mayfly Algorithm (MA) and Binary Bat Algorithm. Our main contribution is to evaluate the effectiveness of various wrapper feature selection techniques and compare the performance of classifiers in generating the optimal variables for the classification of software faults.

The remainder of this study is organized as follows: section 2 presents the previous works related to current research. Used wrapper feature selection techniques are introduced in Section 3. Section 4 presents the proposed methodology followed by the experimental results in Section 5. Section 6 concludes the study with a discussion of our results and prospects for future work.

2 Related Work

Feature selection is one of the most significant tasks of preprocessing data in software fault prediction. The objective of feature selection is to select a subset of optimal features that excludes irrelevant or redundant features. Recently, several studies have proposed different feature selection techniques in SFP. Khoshgoftaar et al. [12] compared different filter feature-ranking selection techniques, namely information gain (IG), ReliefF, chi-squared (CS), symmetrical uncertainty (SU) and gainratio (GR) using 16 fault datasets. They also considered the signal-to-noise ratio (SNR) which is infrequently employed as software metric. Throughout the experimental results, the author suggested IG and SNR achieved the best classification performance across all datasets. However, Khoshgoftaar also recommended that although feature selection can improve the performance of fault prediction models but it has not yet addressed the imbalanced data problem. Wang [27] performed a study to address the problem of which feature elimination methods are stable in case of data change (the deletion or addition of features). They examined various filter-based and wrapper-based feature subset selection techniques using three real-world fault datasets. From the experimental results, they showed that the Correlation-Based Feature Selection (CFS) is the most stable method among different methods. Mohammad et al. [13] demonstrated the combination of feature selection and ensemble learning had a great performance of fault classification. They also concluded that greedy forward selection provided better performance than other methods and average probability ensemble which grouped seven devised classifiers outperformed conventional methods such as random forest and weighted SVMs. Wang [26] proposed a Cluster-based Hybrid Feature Selection that combined Filter and Wrapper methods to reduce the irrelevant and redundant information and enhance the performance of predictive model. Their experimental results indicated that the Cluster-based Hybrid Feature Selection method obtained better performance compared to the other traditional methods in terms of accuracy evaluation measure. Savina Colaco et al. [6] presented a survey of different feature selection techniques specifically filter, wrapper and embedded methods. They found that wrapper methods indicated notable improvement in accuracy for the dataset containing larger features, hence wrapper methods are more popularly applied to obtain optimal features than other methods. Ghost et al. [10] proposed a hybrid of wrapper and filter feature selection based on ant colony optimization. They demonstrated their proposed method outperformed most of the modern methods used for feature selection.

3 Feature Selection

Feature selection is a high dimensionality reduction method by selecting the most important and relevant features, which is the smallest set from a large set of features in the given dataset. There are three types of feature selection methods including the Filter method, Wrapper method and Embedded method. In this study, Wrapper methods are adopted to select relevant features from

the Promise dataset. Additionally, the features of the software fault dataset are defined as software metrics. Therefore, in this section, we concentrate on the software metrics and wrapper-based feature selection.

3.1 Software Metrics

Various kinds of features indicate the characteristic of software. Software metrics are used to evaluate various aspects of software development and software characteristics, such as software quality, complexity, performance, and maintainability. These metrics provide valuable insights into the development process and can help identify areas for improvement.

Software metrics can be broadly divided into four categories: Procedure, Object-Oriented, Hybrid and Miscellaneous Metrics [4, 17]. Procedure software metrics typically focus on the internal characteristics of software, such as its size, complexity, and maintainability. Some examples of traditional software metrics are Line Of Code (LOC), McCabe [16] and Halstead Metrics [11]. Object-oriented (OO) metrics are applied to measure the properties of object oriented software such as Coupling between Objects (CBO), Depth of inheritance tree (DIT), Number of children (NOC), Response for a Class [1]. Software hybrid metrics are a set of software metrics that combine elements of traditional and object-oriented metrics to provide a more comprehensive view of software quality and complexity [15]. Hybrid metrics are designed to measure both the internal and external characteristics of software, such as its functionality, usability, and maintainability. Some common hybrid metrics [25, 7] include Function point analysis (FPA), Maintainability index (MI). Miscellaneous metrics are process metrics [8], change metrics [18], etc. According to the research 's Ruchika Malhotra [15], the most commonly used metrics are recorded in the procedure and object-oriented metrics.

3.2 Genetic Algorithm

Genetic algorithm (GA) is an optimization algorithm that is inspired by natural selection. It is a population-based search algorithm that utilizes the concept of survival of the fittest. The new populations are produced by iterative use of genetic operators on individuals present in the population. The chromosome representation, selection, crossover, mutation, and fitness function computation are the key elements of GA.

3.3 Particle Swarm Optimization

Particle Swarm Optimization was proposed by Kennedy and Eberhart in 1995. This method is inspired by the habits of sharing information from finding food processes of a school of fish or flock of birds that move in a group. In a PSO algorithm, particles (which represent candidate solutions) are flying around, in a multi-dimensional search space. The positions of these particles are adjusted according to the particles' own memories and the best-fit particle of the neighboring particles.[21].

3.4 Whale Optimization Algorithm

Whale optimization algorithm (WOA) is a nature-inspired meta-heuristic optimization algorithm that mimics the hunting behavior of humpback whales. The algorithm is inspired by the bubble-net hunting strategy. Foraging behavior of Humpback whales is called the bubble-net feeding method. Humpback whales prefer to hunt schools of krill or small fishes close to the surface. We can start WOA with a random initialization of the search agents and choose the best, first solution by calculating the fitness of each search agent. Next, the position is updated by basing on equations of Spiral Bubble-Net Feeding Behavior method and then search space constraints are checked and the best solution is updated if there is a better one found. Finally, the algorithm loop is terminated by the maximum number of iterations (default criterion) [3].

3.5 Cuckoo Search

Cuckoo search (CS) is a meta-heuristic algorithm inspired by the bird cuckoo, these are the "Brood parasites" birds. In a nest, each egg represents a solution and the cuckoo egg represents a new and good solution. The obtained solution is a new solution based on the existing one and the modification of some characteristics. In CS algorithm, there are three rules: 1) Each cuckoo lays one egg at a time, and dumps its egg in randomly chosen nest; 2) The best nests with high quality of eggs will carry over to the next generations; 3) The number of available host nests is fixed, and the egg laid by a cuckoo is discovered by the host bird with a probability $p_a \in [0, 1]$. In this case, the host bird can either throw the egg away or abandon the nest, and build a completely new nest [28].

3.6 Mayfly Algorithm

Inspired from the flight behavior and the mating process of mayflies, the Mayfly Algorithm (MA) combines major advantages of swarm intelligence and evolutionary algorithms [29]. In MA, the mayfly swarms would be separated into male and female individuals. Specially, the male mayflies would always be strong and consequently, resulting in a better performance in optimization. And the individuals in MA would update their position according to their current position and velocity the same way as the individuals in PSO. All of the male mayflies and female mayflies would update their positions with the same measure. However, their velocity would be updated in different ways. The velocity of mayflies would be updated according to their current fitness values and the historical best fitness values due to the responsibility of carrying on exploration or exploitation procedures during iterations. The female mayflies would update their velocities with a different style, they would update their velocities based on the male mayflies they want to mate because of a short life cycle within seven days. All of the top half of female and male mayflies would be mated and given a pair of children for every one of them. Their offspring would be randomly evolved

from their parents:

$$offspring_1 = L * male + (1 - L) * female \quad (1)$$

$$offspring_2 = L * female + (1 - L) * male \quad (2)$$

where, L are random numbers in Gauss distribution [9].

3.7 Binary Bat Algorithm

Based on the bat's behaviour, an interesting meta-heuristic optimization technique called Bat Algorithm was developed. Such technique has been developed to behave as a band of bats tracking prey/foods using their capability of echolocation. In order to model this algorithm, some rules had been idealized, as follows: 1) All bats use echolocation to sense distance, and they also "know" the difference between food/prey and background barriers in some magical way; 2) A bat b_i fly randomly with velocity v_i at position x_i with a fixed frequency f_{min} , varying wavelength and loudness A_0 to search for prey. They can automatically adjust the wavelength (or frequency) of their emitted pulses and adjust the rate of pulse emission $r \in [0, 1]$, depending on the proximity of their target; 3) Although the loudness can vary in many ways, the loudness varies from a large (positive) was assumed that A_0 to a minimum constant value A_{min} . In feature selection, since the problem is to select or not a given feature, the bat's position is then represented by binary vectors restricting the new bat's position to only binary values using a sigmoid function (BBA) [20].

3.8 Feature Selection Details

In order to select features effectively, we taked advantage of feature selection module of sklearn library developed in Python that provides many machine learning algorithms. For the GA, we use 10 number of agents, cross and mutation probability are set to 0.7 and 0.1 correspondingly. For the rest, just one agents is used in selection process. Finally, $\log_2(S)$ is set to max iterations of each method, where S is the number of samples.

4 Methodology

4.1 Proposed Approach

As we mentioned in introduction, software fault prediction is a vital procedure in software engineering and also depends on historical datasets that have been collected from previous software projects. And this process still has two major challenges including high dimensional data and imbalanced classes. Therefore we propose an approach to examine the the effectiveness of six different Wrapper methods for selecting the subset of optimal features. Then the variant of generative adversarial network - VanillaGAN [23] is applied to generate synthetic

instances of faulty modules to balance the ratio of faulty and non-faulty modules in the fault datasets.

The overall approach is shown in Fig.1. In data preprocessing stage, we fill the missing values and apply Z-normalization for data normalization. Next, we apply six different wrapper methods, namely GA, PSO, WOA, CS, MA and BBA to select the most important and relevant features for reducing high dimension features. The datasets with optimal features are used to train the VanillaGAN network to overcome the imbalanced data. Finally, these training datasets, and testing datasets are used to train using machine learning techniques, e.g K-Nearest Neighbors (KNN), Random Forest (RF), Decision Tree (DT), Naïve Bayes (NB) and Logistic Regression (LR). We use precision, recall, F1-score and AUC as measures to evaluate the performance of different wrapper-based feature selection methods.

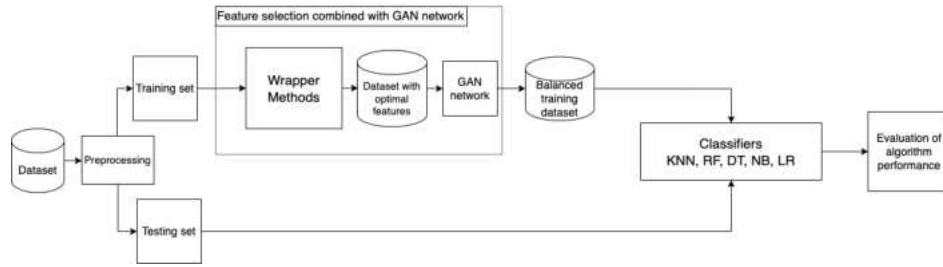


Fig. 1. Flow diagram of proposed approach

4.2 VanillaGAN for Handling imbalanced data

We use Vanilla Gan, which is the simplest version of GAN. The generator network learn the probability of the training set by mapping the noise z which is drawn independent and identically distributed (i.i.d) from $\mathbb{N}(0, 0.01)$ and added to both real and synthetic data to the probability distribution of training samples. Then, synthetic samples that are closer possible to real samples were generated by the generator network. It uses backpropagation to train both models. In Vanilla GAN, we use neural networks to approximate complex, high-dimensional distributions for both Gen and Dis . The discriminator training is done by minimizing its prediction error, whereas the generator is trained by maximizing the prediction error by the discriminator. This can be formalized as given follows:

$$\min_{(\theta_{Gen})} \max_{(\theta_{Dis})} (E_{x \sim p_D} [\log Dis(x)] + E_{z \sim p_z} [\log(1 - Dis(Gen(z)))] \quad (3)$$

where p_D is the real data distribution, p_z is the prior distribution of the generative network, and θ_{Gen} and θ_{Dis} are the parameters of the generator (Gen)

and discriminator (Dis). Given a strong discriminator, the generator’s goal is achieved if p_D , the generator’s distribution over x , equals p_z , the real distribution, which means that the Jensen–Shannon Divergence (JSD) is minimized [23].

4.3 Dataset

In this study, we used five sub-datasets collected from PROMISE repository which are collections of publicly available datasets. These resources are adopted to serve researchers in building predictive software models and the software engineering community at large. In terms of software fault prediction (SFP) problems, the datasets provide independent variables which are the different source code metrics such as line count of code (loc), coupling between objects (CBO), etc., and dependent variable in faulty and non-faulty with labels (0) and (1) correspondingly. The details of the used datasets are given in Table 1.

Table 1. Description of used software fault datasets

Dataset	Release	Instances	Metric	Faulty Instances	Imbalanced ratio (%)
PROMISE	CM1	505	40	48	9.504
	KC1	2107	21	325	15.424
	KC2	522	20	107	20.498
	PC1	1107	40	76	6.865
	JM1	10885	20	2106	19.347

4.4 Evaluation Measurement

Performance evaluation measures such as Precision, Recall, F1-score and AUC are used to evaluate how the wrapper feature selection algorithms affect the performance of prediction models. Precision is the ratio between the True Positive and all the Positive, meanwhile Recall is the measure of our model correctly identifying True Positive. In software fault prediction problems, Precision would be the measure of instances that our model correctly identifies as faulty labels out of all instances actually are faulty labels. For all instances which actually are faulty labels, Recall tells us how many instances our model correctly identifies as faulty labels. These metrics are defined by the following:

$$Precision(P) = \frac{TP}{TP + TF}, Recall(R) = \frac{TP}{TP + FN} \quad (4)$$

where TP = True positive, FP = False positive, FN = False negative. From two values of Precision and Recall, we can calculate additional metrics that are F1-score and AUC:

$$F1 - score = \frac{2 * P * R}{P + R} \quad (5)$$

The F1-score is the harmonic mean of the Precision and Recall. The highest possible value of an F-score is 1.0, indicating perfect precision and recall, and the lowest possible value is 0, if either precision or recall are zero. Besides that, AUC calculates the area under the receiver operating characteristics(ROC). This metric represents the probability that a random positive instance to the right of a random negative instance. The higher the probability is, the more perfect the prediction is.

5 Experimental Result

We use a separate training and testing dataset for each repetition of 10-time experiments; the final result is the average of the 10 iterations. The result has been reported in Table 2 and Table 3, including five machine learning techniques for the average result of five datasets of each Wrapper method with regard to used performance measures.

Dataset	Performance Evaluation	GA+VanillaGAN					PSO+VanillaGAN					WOA + VanillaGAN					Without FS				
		KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR
CM1	Precision	0.808	0.790	0.804	0.750	0.859	0.806	0.813	0.81	0.805	0.833	0.821	0.782	0.808	0.803	0.829	0.792	0.780	0.801	0.749	0.800
	Recall	0.726	0.829	0.811	0.734	0.786	0.740	0.860	0.849	0.863	0.686	0.754	0.860	0.837	0.866	0.706	0.707	0.825	0.811	0.803	0.876
	F1-score	0.758	0.808	0.806	0.741	0.820	0.768	0.832	0.827	0.831	0.738	0.783	0.819	0.819	0.833	0.752	0.747	0.801	0.805	0.775	0.826
	AUC	0.617	0.688	0.646	0.509	0.690	0.689	0.740	0.645	0.524	0.687	0.675	0.729	0.617	0.543	0.720	0.608	0.683	0.637	0.506	0.686
KC1	Precision	0.813	0.814	0.793	0.831	0.824	0.822	0.829	0.818	0.837	0.821	0.825	0.832	0.818	0.834	0.824	0.810	0.819	0.816	0.815	0.801
	Recall	0.809	0.833	0.828	0.778	0.670	0.831	0.855	0.847	0.657	0.666	0.836	0.854	0.852	0.706	0.697	0.802	0.830	0.819	0.697	0.656
	F1-score	0.811	0.818	0.804	0.797	0.713	0.825	0.817	0.819	0.701	0.710	0.828	0.816	0.821	0.741	0.735	0.805	0.810	0.813	0.751	0.701
	AUC	0.726	0.789	0.769	0.783	0.755	0.746	0.817	0.795	0.769	0.744	0.741	0.804	0.790	0.780	0.766	0.715	0.785	0.763	0.764	0.799
KC2	Precision	0.869	0.876	0.882	0.879	0.884	0.881	0.879	0.870	0.871	0.876	0.874	0.879	0.876	0.884	0.872	0.866	0.863	0.866	0.872	0.870
	Recall	0.841	0.875	0.862	0.745	0.693	0.866	0.924	0.913	0.900	0.789	0.866	0.922	0.913	0.907	0.793	0.817	0.884	0.857	0.879	0.769
	F1-score	0.853	0.875	0.870	0.793	0.761	0.872	0.894	0.889	0.884	0.826	0.869	0.894	0.892	0.893	0.827	0.840	0.870	0.861	0.875	0.816
	AUC	0.719	0.76	0.721	0.707	0.707	0.739	0.785	0.715	0.672	0.658	0.729	0.783	0.727	0.671	0.635	0.710	0.759	0.701	0.67	0.637
PC1	Precision	0.817	0.818	0.788	0.819	0.828	0.797	0.811	0.801	0.820	0.824	0.814	0.808	0.798	0.806	0.832	0.812	0.807	0.782	0.81	0.819
	Recall	0.798	0.813	0.792	0.828	0.755	0.783	0.825	0.819	0.823	0.749	0.798	0.825	0.817	0.817	0.796	0.727	0.810	0.803	0.82	0.722
	F1-score	0.804	0.811	0.784	0.816	0.774	0.789	0.811	0.789	0.818	0.769	0.800	0.806	0.797	0.808	0.807	0.767	0.808	0.792	0.814	0.767
	AUC	0.772	0.804	0.738	0.817	0.820	0.760	0.826	0.817	0.818	0.817	0.788	0.806	0.798	0.772	0.819	0.770	0.767	0.737	0.74	0.809
JM1	Precision	0.747	0.766	0.758	0.768	0.759	0.747	0.762	0.757	0.764	0.750	0.742	0.749	0.747	0.762	0.761	0.738	0.747	0.742	0.756	0.740
	Recall	0.768	0.809	0.806	0.796	0.704	0.774	0.807	0.805	0.785	0.555	0.768	0.770	0.776	0.796	0.591	0.766	0.760	0.777	0.785	0.511
	F1-score	0.756	0.760	0.755	0.777	0.725	0.758	0.756	0.754	0.771	0.595	0.753	0.753	0.751	0.771	0.631	0.751	0.759	0.753	0.770	0.541
	AUC	0.658	0.708	0.702	0.663	0.668	0.659	0.708	0.700	0.674	0.643	0.651	0.69	0.684	0.68	0.674	0.674	0.679	0.681	0.642	0.638
Average results																					
	Precision	0.811	0.813	0.805	0.809	0.831	0.811	0.819	0.811	0.819	0.821	0.815	0.810	0.809	0.818	0.824	0.804	0.803	0.801	0.800	0.806
	Recall	0.788	0.832	0.820	0.776	0.722	0.799	0.854	0.847	0.806	0.689	0.804	0.846	0.839	0.818	0.717	0.764	0.822	0.813	0.797	0.707
	F1-score	0.796	0.814	0.804	0.785	0.759	0.802	0.822	0.816	0.801	0.728	0.807	0.818	0.816	0.809	0.750	0.782	0.810	0.805	0.797	0.730
	AUC	0.698	0.750	0.715	0.696	0.728	0.719	0.775	0.734	0.691	0.710	0.717	0.762	0.723	0.689	0.723	0.695	0.735	0.704	0.664	0.714

FS: Feature Selection

Table 2. Comparison results of presented Wrapper feature selection techniques combined with Vanilla GAN for all the used dataset.

For the Precision measure, values are almost higher than 0.80 for all the techniques except JM1 dataset, the highest value is 0.884 for the PC1 dataset

Dataset	Performance Evaluation	MA+VanillaGAN					CS+VanillaGAN					BBA+VanillaGAN					Without FS				
		KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR	KNN	RF	DT	NB	LR
CM1	Precision	0.814	0.809	0.804	0.787	0.844	0.819	0.818	0.817	0.785	0.847	0.82	0.794	0.798	0.789	0.836	0.792	0.780	0.801	0.749	0.800
	Recall	0.737	0.866	0.843	0.854	0.726	0.731	0.874	0.846	0.854	0.723	0.771	0.854	0.834	0.851	0.72	0.707	0.825	0.811	0.803	0.876
	F1-score	0.769	0.83	0.82	0.818	0.768	0.766	0.839	0.828	0.816	0.766	0.792	0.82	0.813	0.817	0.763	0.747	0.801	0.805	0.775	0.826
	AUC	0.676	0.708	0.644	0.542	0.727	0.683	0.744	0.671	0.568	0.715	0.686	0.727	0.623	0.538	0.689	0.608	0.683	0.637	0.506	0.686
KC1	Precision	0.818	0.84	0.825	0.828	0.824	0.829	0.823	0.815	0.833	0.819	0.817	0.834	0.817	0.813	0.824	0.810	0.819	0.816	0.815	0.801
	Recall	0.829	0.859	0.853	0.721	0.688	0.829	0.853	0.848	0.666	0.673	0.827	0.858	0.849	0.663	0.673	0.802	0.830	0.819	0.697	0.656
	F1-score	0.822	0.827	0.82	0.752	0.728	0.818	0.814	0.813	0.709	0.715	0.821	0.827	0.818	0.694	0.716	0.805	0.810	0.813	0.751	0.701
	AUC	0.742	0.807	0.782	0.782	0.771	0.742	0.805	0.779	0.776	0.747	0.743	0.812	0.784	0.767	0.757	0.715	0.785	0.763	0.764	0.799
KC2	Precision	0.876	0.879	0.871	0.879	0.878	0.87	0.879	0.87	0.878	0.874	0.882	0.877	0.87	0.879	0.879	0.866	0.863	0.866	0.872	0.870
	Recall	0.859	0.922	0.914	0.904	0.789	0.854	0.924	0.913	0.897	0.779	0.868	0.924	0.909	0.897	0.786	0.817	0.884	0.857	0.879	0.769
	F1-score	0.867	0.894	0.89	0.89	0.827	0.861	0.894	0.889	0.887	0.818	0.874	0.895	0.887	0.886	0.824	0.840	0.870	0.861	0.875	0.816
	AUC	0.717	0.789	0.716	0.665	0.659	0.716	0.79	0.72	0.675	0.652	0.736	0.801	0.719	0.669	0.652	0.710	0.759	0.701	0.67	0.637
PC1	Precision	0.797	0.807	0.782	0.817	0.823	0.783	0.824	0.829	0.826	0.83	0.827	0.815	0.785	0.812	0.828	0.812	0.807	0.782	0.81	0.819
	Recall	0.774	0.823	0.794	0.825	0.747	0.758	0.819	0.825	0.832	0.766	0.809	0.826	0.806	0.821	0.787	0.727	0.810	0.803	0.82	0.722
	F1-score	0.782	0.811	0.785	0.815	0.768	0.767	0.818	0.824	0.822	0.784	0.815	0.816	0.792	0.81	0.798	0.767	0.808	0.792	0.814	0.767
	AUC	0.75	0.787	0.735	0.815	0.815	0.747	0.805	0.783	0.826	0.819	0.806	0.808	0.754	0.781	0.812	0.770	0.767	0.737	0.74	0.809
JM1	Precision	0.747	0.742	0.732	0.756	0.757	0.76	0.76	0.757	0.767	0.761	0.76	0.764	0.764	0.767	0.745	0.738	0.747	0.742	0.756	0.740
	Recall	0.778	0.757	0.762	0.777	0.593	0.787	0.806	0.803	0.793	0.725	0.784	0.8	0.801	0.803	0.604	0.766	0.760	0.777	0.785	0.511
	F1-score	0.756	0.745	0.741	0.764	0.63	0.769	0.758	0.759	0.776	0.74	0.769	0.765	0.76	0.774	0.642	0.751	0.759	0.753	0.770	0.541
	AUC	0.654	0.679	0.668	0.655	0.669	0.666	0.72	0.709	0.663	0.672	0.686	0.709	0.703	0.682	0.651	0.674	0.679	0.681	0.642	0.638
Average results																					
FS: Feature Selection	Precision	0.81	0.815	0.803	0.813	0.825	0.812	0.821	0.818	0.818	0.826	0.821	0.817	0.807	0.812	0.822	0.804	0.803	0.801	0.800	0.806
	Recall	0.795	0.845	0.833	0.816	0.709	0.792	0.855	0.847	0.808	0.733	0.812	0.852	0.84	0.807	0.714	0.764	0.822	0.813	0.797	0.707
	F1-score	0.799	0.821	0.811	0.808	0.744	0.796	0.825	0.823	0.802	0.765	0.814	0.825	0.814	0.796	0.749	0.782	0.810	0.805	0.797	0.730
	AUC	0.708	0.754	0.709	0.692	0.728	0.711	0.773	0.732	0.702	0.721	0.731	0.771	0.717	0.687	0.712	0.695	0.735	0.704	0.664	0.714

FS: Feature Selection

Table 3. Comparison results of presented Wrapper feature selection techniques combined with Vanilla GAN for all the used dataset (cont).

in GA and WOA method, and the lowest value is 0.732 for the JM1 dataset in MA method. For the Recall measure, values almost higher than 0.80, PSO and CS obtained the highest value of 0.924 for PC1 and the lowest value is 0.591 for JM1 dataset in WOA method. For the F1-score, BBA reached the highest value of 0.895 for the PC1 dataset, PSO and CS reached a value of 0.894, which is also good, and the lowest value is 0.40 for the CM1 dataset. In terms of AUC measure, for most of the cases, values are greater than 0.700 in CM1, PC1, KC1 and KC2. The highest value of AUC is 0.826 for KC2 in PSO method and the lowest value is 0.509 for CM1. Additionally, the performance of the classifiers based on wrapper feature selection methods was better than when no feature selection methods are applied.

From the average results, it can be observed that PSO with Vanilla GAN and Cuckoo Search with VanillaGAN significantly produced highlighted values, which are higher than others for most of the cases, followed by Mayfly Algorithm with Vanilla GAN. Moreover, it is clear that the RF technique produces the best performance for most cases among the used wrapper methods for evaluation measures.

6 Conclusion

The performance of software fault prediction (SFP) is affected by two major factors that are high dimensional features and imbalanced classes. One of the common ways to overcome these problem can be a hybrid feature selection method combined with data sampling that generate balanced dataset with optimal features.

For the high dimension reduction, this paper presented six Wrapper methods that are Genetic Algorithm, the Particle Swarm Optimization, Whale Optimization Algorithm, Cuckoo Search, Mayfly Algorithm and Binary Bat Algorithm to select important and relevant features from Promise dataset. Then the Vanilla GAN was trained with the collected dataset to generate synthetic instances of minority class to handle imbalanced data. Afterward, five machine learning techniques were applied on balanced data in SFP. Experiment results show us a significant achievement of combination of wrapper feature selection and GAN network for SFP, specifically the Particle Swarm Optimization with Vanilla GAN and Cuckoo Search with VanillaGAN produced a good performance on most of the dataset, followed by Mayfly Algorithm with Vanilla GAN.

In the future work, we intend to investigate on ensemble learning for feature selection and several variations of GAN for handling the imbalanced dataset in SFP.

References

1. Aggarwal, K., Singh, Y., Kaur, A., Malhotra, R.: Empirical study of object-oriented metrics. *J. Object Technol.* **5**(8), 149–173 (2006)
2. Arisholm, E., Briand, L.C., Johannessen, E.B.: A systematic and comprehensive investigation of methods to build and evaluate fault prediction models. *Journal of Systems and Software* **83**(1), 2–17 (2010)
3. Brodzicki, A.; Piekarski, M.J.K.J.: The whale optimization algorithm approach for deep neural networks. *Sensors* **21**, 8003 (2021)
4. Caglayan, B., Tosun, A., Miranskyy, A., Bener, A., Ruffolo, N.: Usage of multiple prediction models based on defect categories. In: *Proceedings of the 6th International Conference on Predictive Models in Software Engineering*. pp. 1–9 (2010)
5. Chandrashekar, G., Sahin, F.: A survey on feature selection methods. *Computers & Electrical Engineering* **40**(1), 16–28 (2014)
6. Colaco, S., Kumar, S., Tamang, A., Biju, V.G.: A review on feature selection algorithms. *Emerging Research in Computing, Information, Communication and Applications: ERCICA 2018, Volume 2* pp. 133–153 (2019)
7. De Carvalho, A.B., Pozo, A., Vergilio, S.R.: A symbolic fault-prediction model based on multiobjective particle swarm optimization. *Journal of Systems and Software* **83**(5), 868–882 (2010)
8. Elish, K.O., Elish, M.O.: Predicting defect-prone software modules using support vector machines. *Journal of Systems and Software* **81**(5), 649–660 (2008)
9. Gao, Z.M., Zhao, J., Li, S.R., Hu, Y.R.: The improved mayfly optimization algorithm. *Journal of Physics: Conference Series* **1684**, 012077 (11 2020). <https://doi.org/10.1088/1742-6596/1684/1/012077>
10. Ghosh, M., Guha, R., Sarkar, R., Abraham, A.: A wrapper-filter feature selection technique based on ant colony optimization. *Neural Computing and Applications* **32**, 7839–7857 (2020)
11. Halstead, M.H.: *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc. (1977)
12. Khoshgoftaar, T.M., Gao, K., Napolitano, A.: An empirical study of feature ranking techniques for software quality prediction. *Int. J. Softw. Eng. Knowl. Eng.* **22**, 161–183 (2012)

13. Laradji, I.H., Alshayeb, M., Ghouti, L.: Software defect prediction using ensemble learning on selected features. *Information and Software Technology* **58**, 388–402 (2015)
14. Lessmann, S., Baesens, B., Mues, C., Pietsch, S.: Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE transactions on software engineering* **34**(4), 485–496 (2008)
15. Malhotra, R.: A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing* **27**, 504–518 (2015)
16. McCabe, T.J.: A complexity measure. *IEEE Transactions on software Engineering* (4), 308–320 (1976)
17. Meiliana, Karim, S., Warnars, H.L.H.S., Gaol, F.L., Abdurachman, E., Soewito, B.: Software metrics for fault prediction using machine learning approaches: A literature review with promise repository dataset. In: 2017 IEEE International Conference on Cybernetics and Computational Intelligence (CyberneticsCom). pp. 19–23 (2017). <https://doi.org/10.1109/CYBERNETICSCOM.2017.8311708>
18. Moser, R., Pedrycz, W., Succi, G.: A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. In: Proceedings of the 30th international conference on Software engineering. pp. 181–190 (2008)
19. Moslehi, F., Haeri, A.: A novel hybrid wrapper-filter approach based on genetic algorithm, particle swarm optimization for feature subset selection. *Journal of Ambient Intelligence and Humanized Computing* **11**, 1105–1127 (2020)
20. Nakamura, R., Pereira, L., Costa, K., Rodrigues, D., Papa, J., Yang, X.S.: Bba: A binary bat algorithm for feature selection. pp. 291–297 (08 2012). <https://doi.org/10.1109/SIBGRAPI.2012.47>
21. Ovat, F., Anyandi, A..J.: The particle swarm optimization (pso) algorithm application – a review. *Global Journal of Engineering and Technology Advances* **3**, 001–006 (06 2020)
22. Pandey, S.K., Mishra, R.B., Tripathi, A.K.: Machine learning based methods for software fault prediction: A survey. *Expert Systems with Applications* **172**, 114595 (2021)
23. Rathore, S.S., Chouhan, S.S., Jain, D.K., Vachhani, A.G.: Generative oversampling methods for handling imbalanced data in software fault prediction. *IEEE Transactions on Reliability* **71**(2) (2022)
24. Shepperd, M., Song, Q., Sun, Z., Mair, C.: Data quality: Some comments on the nasa software defect datasets. *IEEE Transactions on Software Engineering* **39**(9), 1208–1215 (2013)
25. Turhan, B., Kocak, G., Bener, A.: Software defect prediction using call graph based ranking (cgbr) framework. In: 2008 34th Euromicro Conference Software Engineering and Advanced Applications. pp. 191–198. IEEE (2008)
26. Wang, F., Ai, J., Zou, Z.: A cluster-based hybrid feature selection method for defect prediction. In: 2019 IEEE 19th International Conference on Software Quality, Reliability and Security (QRS). pp. 1–9. IEEE (2019)
27. Wang, H., Khoshgoftaar, T.M., Napolitano, A.: Stability of filter-and wrapper-based software metric selection techniques. In: Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014). pp. 309–314. IEEE (2014)
28. Yang, X.S., Deb, S.: Cuckoo search via lévy flights. In: 2009 World Congress on Nature Biologically Inspired Computing (NaBIC). pp. 210–214 (2009)
29. Zervoudakis, K., Tsafarakis, S.: A mayfly optimization algorithm. *Comput. Ind. Eng.* **145**, 106559 (2020)